

POSTGIS IN ARABIC

By Ahmad Saleh

April 2026

These videos and the training manual in PostGIS to fill a major gap in Arabic content. My goal is to share technical knowledge that many Arab developers are missing.

I believe access to information in our language makes learning easier and more effective.

I want to empower the Arab community to confidently use GIS and spatial database technologies.

This work is a step toward building a strong and valuable Arabic resource in this field.

This manual is designed to work alongside the videos for the best learning experience.

It's not structured as a textbook—it's meant to be practical notes to follow while watching and learning..

Links :

GitHub link for the data and documents

<https://github.com/ahmadsaleh009/postgis-spatial-project-arabic>

POSTGIS workshop [Introduction to PostGIS](#)

<https://postgis.net/workshops/postgis-intro/index.html>

Download Postgres

<https://www.postgresql.org/download/>

PostgreSQL Quick Start (Core SQL)

Quick start

Create database sales;

```
create table customers (name text , phonenumber text)
```

```
insert into customers (name,phonenumber)  
values ('ahmad','123345678')
```

```
select * from customers
```

Think of **schemas** as folders inside a database.

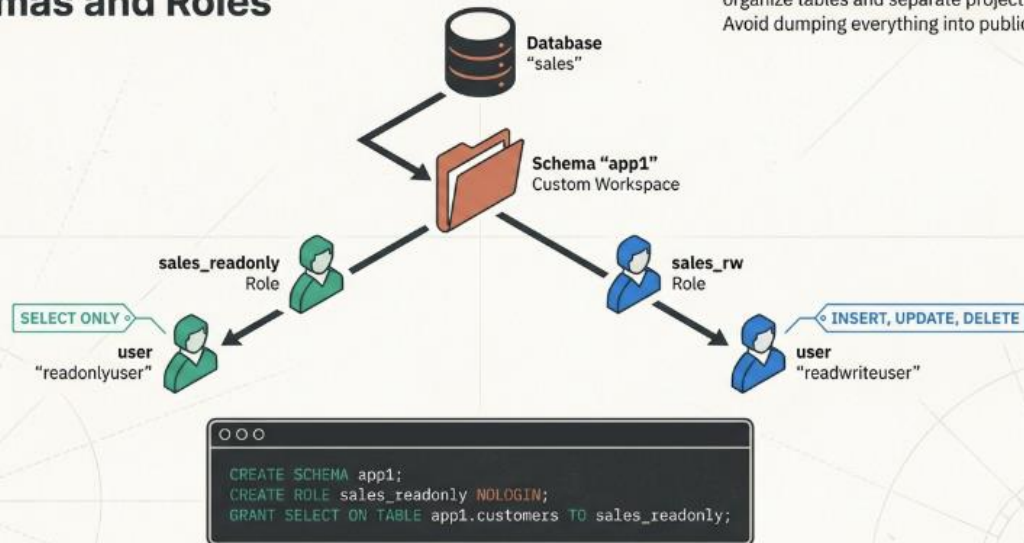
Why Schemas Exist

Schemas help you:

- ✓ Organize tables
- ✓ Separate projects
- ✓ Control access
- ✓ Backup/restore groups of tables

The Foundation: Schemas and Roles

Before drawing the map, prepare the canvas. Treat Schemas as folders to organize tables and separate projects. Avoid dumping everything into public.



CREATE SCHEMA myfirstschema;

create schema app1;

Now create your tables in this schema, then we will add users , roles and give permissions to the roles for users to access these tables

CREATE USER adam WITH LOGIN PASSWORD '123';

grant CONNECT ON database mynewdb to adam;

GRANT USAGE ON SCHEMA myfirstschema TO adam;

-- read only

GRANT SELECT ON TABLE myfirstschema.customers to adam

-- read and write.

GRANT SELECT, INSERT, UPDATE, DELETE
ON TABLE myfirstschema.customers to adam

Better method is you use roles , add users inside the roles and give the roles the permissions

-- roles

```
CREATE ROLE sales_readonly NOLOGIN;  
CREATE ROLE sales_rw NOLOGIN;
```

--create users

```
CREATE USER readonlyuser WITH LOGIN PASSWORD '123';  
CREATE USER readwriteuser WITH LOGIN PASSWORD '123';
```

```
GRANT sales_readonly TO readonlyuser;  
GRANT sales_rw TO readwriteuser;
```

```
GRANT CONNECT ON DATABASE mynewdb TO sales_readonly, sales_rw;
```

```
GRANT USAGE ON SCHEMA public TO sales_readonly, sales_rw;
```

```
GRANT SELECT  
ON TABLE myfirstschema.customers  
TO sales_readonly;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE  
ON TABLE myfirstschema.customers  
TO sales_rw;
```

Common Column Types (you'll actually use)

Category	Modern Type / Pattern	Example Syntax	Use Case
Primary Key (numeric)	BIGINT IDENTITY	id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY	Internal IDs
Primary Key (public)	UUID	id UUID DEFAULT gen_random_uuid()	Public/shared IDs
Text	TEXT	name TEXT	Names, descriptions
Money / Decimal	NUMERIC(12,2)	price NUMERIC(12,2)	Currency
Integer	INTEGER	qty INTEGER	Counts
Boolean	BOOLEAN	is_active BOOLEAN DEFAULT true	Flags
Timestamp	TIMESTAMPTZ	created_at TIMESTAMPTZ DEFAULT now()	Time tracking
Date only	DATE	birth_date DATE	Calendar date
Status	TEXT + CHECK	status TEXT CHECK (status IN (...))	Controlled values
Flexible Data	JSONB	meta JSONB DEFAULT '{}'	Extra attributes
Small Numbers	SMALLINT	rating SMALLINT	Small ranges
Arrays	TEXT[]	tags TEXT[]	Small lists
Binary Data	BYTEA	file BYTEA	Small files
Range Types	TSTZRANGE	valid_range TSTZRANGE	History tracking

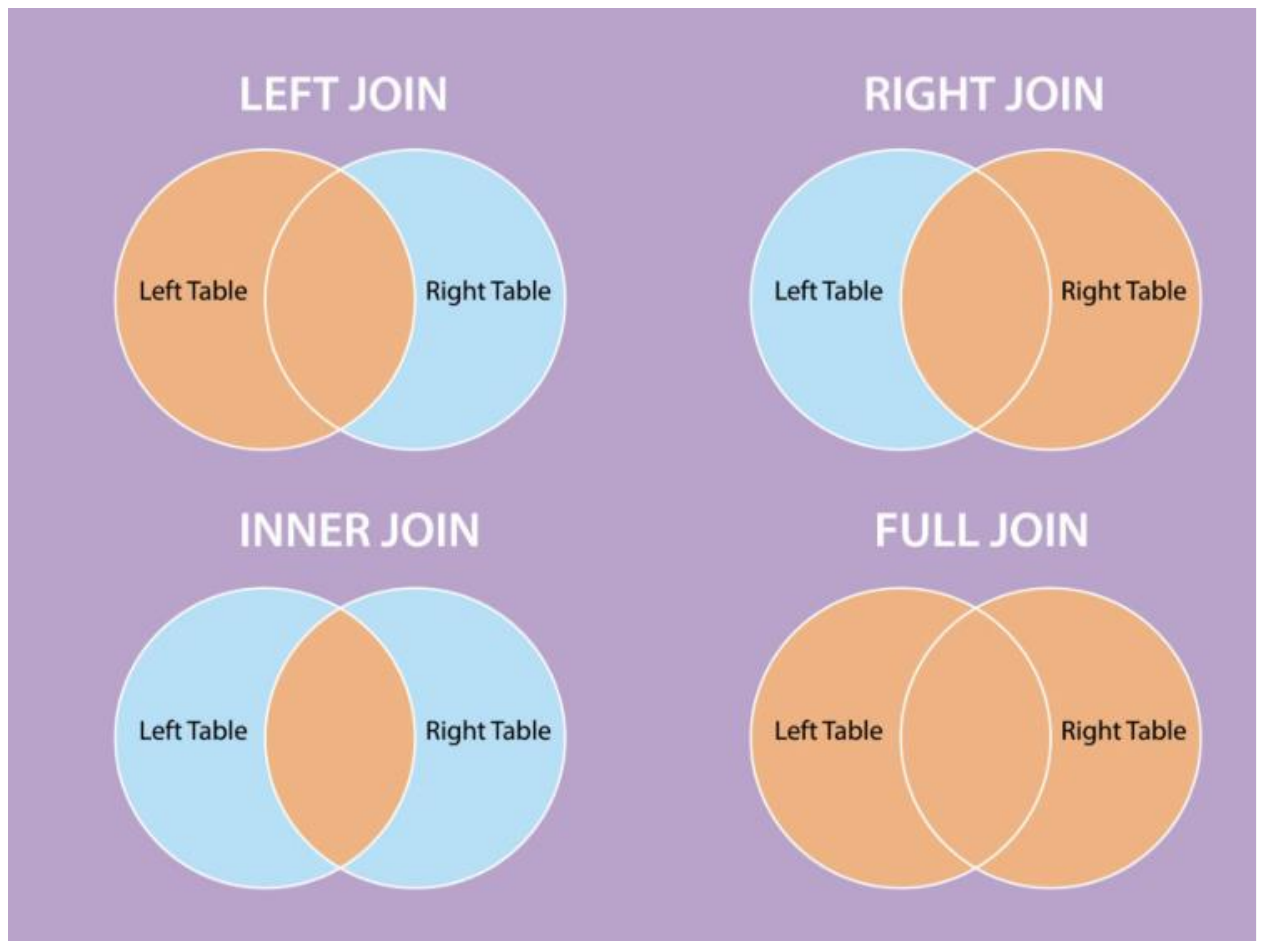
SQL JOIN Types

INNER → matches only

LEFT → everything from left table

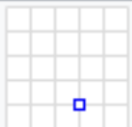
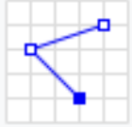
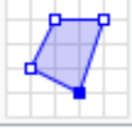
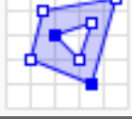
RIGHT → everything from right table

FULL → everything from both



POSTGIS Intro

- now this is the basic stuff , we need to add new table but spatial activated using the postgis extention.
- `CREATE EXTENSION IF NOT EXISTS postgis;`
- `SELECT postgis_full_version();`
-
- `ST_GeomFromText` creates a **geometry object** from **WKT (Well-Known Text)**.
- WKT is a standard text way to describe spatial shapes using coordinates.

Type	Examples	
Point		<code>POINT (30 10)</code>
LineString		<code>LINESTRING (30 10, 10 30, 40 40)</code>
Polygon		<code>POLYGON ((30 10, 40 40, 20 40, 10 20, 30 10))</code>
		<code>POLYGON ((35 10, 45 45, 15 40, 10 20, 35 10), (20 30, 35 35, 30 20, 20 30))</code>

Turning the Database into a Map

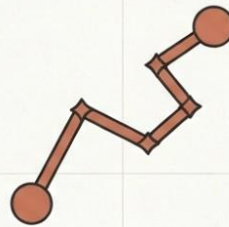
The 'On' Switch: **CREATE EXTENSION postgis;**



POINT

A single location (POIs).

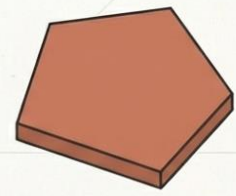
```
ST_GeomFromText('POINT(-78 40)')
```



LINESTRING

A path or road.

```
ST_GeomFromText('LINESTRING(-78 40, -80 45)')
```



POLYGON

A boundary or neighborhood.

```
ST_GeomFromText('POLYGON((-78 40, -80 45, -81 50, -78 40))')
```

-
- Geometry to text use `ST_GeomAsText(wkt,srid)`
- Or you can use `ST_SetSRID (ST_makepoint(lat, long),srid)`
- `select ST_GeomFromText('POINT(-78 40)')`
- `select st_geomfromtext ('Linestring (-78 40 , -80 45)')`
- `select st_geomfromtext('polygon ((-78 40 , -80 45, -81 50 , -78 40))')`
-
- now lets create the table testgeo and add column
- `create table testgeo (name text,geo geometry)`
- `insert into testgeo (name,geo) values ('point1','POINT(-78 40)')`
- `select * from testgeo`
- another method to add data points but using `st_makepoint` , and also use `st_setSRID`
- `insert into testgeo (name,geo) values ('point2',(ST_SetSRID (ST_makepoint(-77, 39),4326)))`
- add polygon
- `insert into testgeo (name,geo) values ('poly',ST_SetSRID(ST_MakePolygon(ST_GeomFromText('LINESTRING(-78 40,-79 39,-79 42, -78 40)'),4326))`
- add line
- `insert into testgeo (name,geo) values ('line',ST_SetSRID(st_makeline(st_point(-77, 39),st_point(-78,40)),4326))`
- now lets explore what we did ., the below line shows list of all tables in the db that has geometry column , notice the type of the geometry column for testgeo table !

- `select * from geometry_columns`
- use geometry type to show the type to geometry `select name, ST_GeometryType(geo) from testgeo`
- use ST_Astext to return the geometry as text from geometry object `ST_AsText`
- `select name ,ST_AsText(geo) from testgeo`
- if we have a geometry that means we can the built in functions geometry functions , so what is the length of the line , whats is the area for the polygon , what is the lat and what is the lat/long of a point ?
- -- find me all the lat and long for the point named point1
`select name, ST_x(geo) as x ,ST_y(geo) as y from testgeo where name like '%point1%'`
- calculate the area of all polygons in the geotest table
- `select name, ST_area(geo) from testgeo where ST_geometrytype(geo)='ST_Polygon'`
- - calculate the length of all line in the geotest table using `ST_Length`

```
select name, ST_length(geo) as length
```

```
from testgeo
```

```
where ST_geometrytype(geo)='ST_LineString'
```

- lets look again at test geo `select * from testgeo`
- If the data are in “geographics” (EPSG:4326) then it will place a base map underneath the data. -- why it only displays point 2 .
- To see a map with your output, wrap it in
- -- lets see what is the spatial ref for items in the testgeo

```
select name, st_srid(geo) srid, st_geometrytype(geo) geometry
```

```
from testgeo
```

- how to change it , if you now that your data is in 4326 but you forgot define it --Your data is already longitude/latitude, but SRID = 0.

```
UPDATE testgeo
```

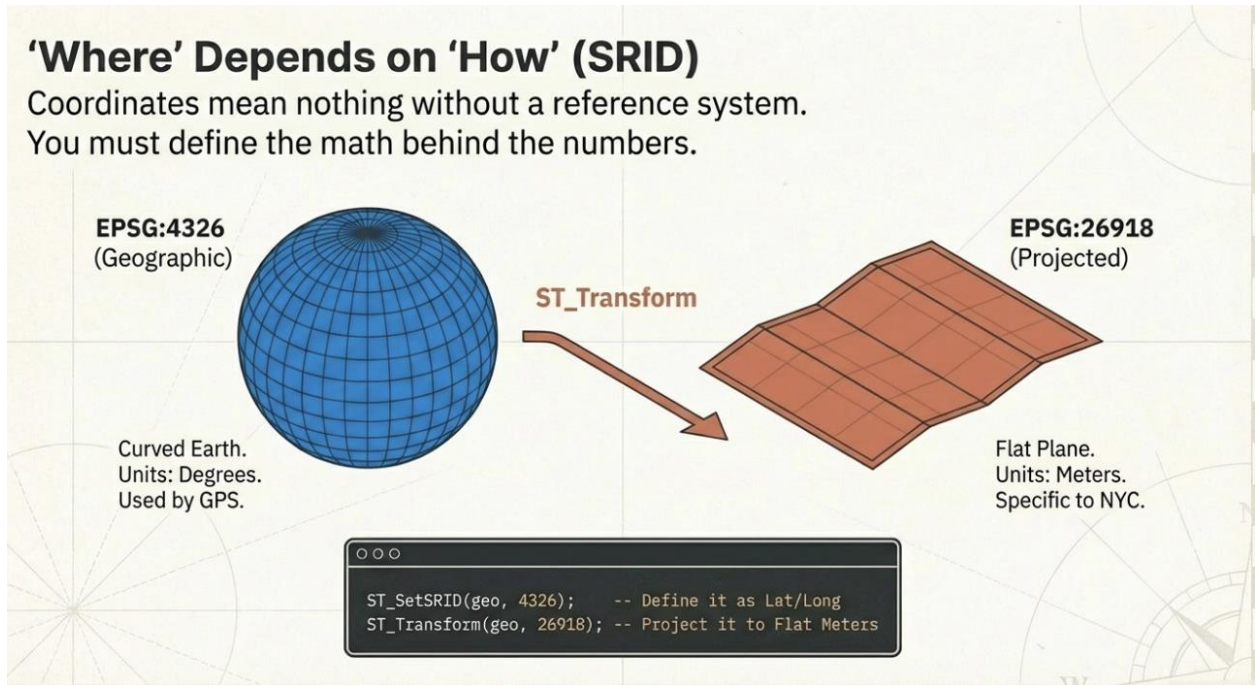
```
SET geo = ST_SetSRID(geo, 4326)
```

```
WHERE ST_SRID(geo) = 0;

select st_SRID(geo) from testgeo

where name='point1'
```

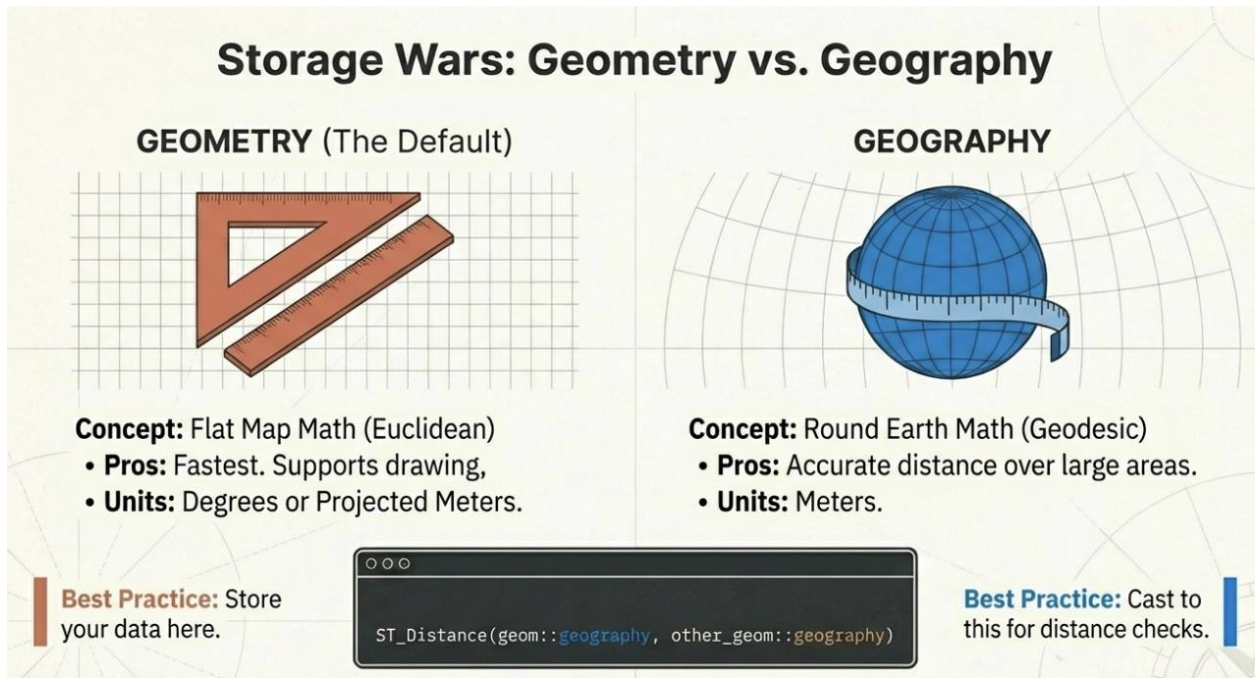
- Coordinates change numerically



Geometry vs Geography ! which one to use ?

create table geographytable (name text,geom geography)

create table geometrytable (name text ,geom geometry)



- **geometry** = flat map math, units depends on the projection, faster than geography good for Maps Drawing shapes, Spatial relationships Indexing, supports advanced spatial ops
- **geography** = real Earth math \ earth is round \ distance in meters \ Coordinates are lon/lat Distance calculations
- Buffers in meters > "Within X km" > delivery radii > Real-world measurements
- Same data. Very different math.
- SRID tells PostGIS:

"How should I interpret these numbers?"

- geometry:

"I will calculate using whatever coordinate system you gave me."

- geography:

"I know this is Earth. I will calculate using spherical math."

- Best-practice pattern (very common) Store data as geometry, calculate with geography:

```
geom geometry(Point, 4326)
```

```
select st_length (geo::geography)
from testgeo
where name ='line'
```

- Geometry is for where things are.
- Geography is for how far apart they are.
- Most spatial algorithms assume a flat coordinate space, which geometry provides. Because of that, almost all ST_ functions are implemented for geometry first.
- PostGIS reprojects geometry from one CRS to another
- ST_Transform(geom, 4326)

```
select name, st_x(geo) old_x, st_y(geo) old_y,
st_x(st_transform(geo, 26918)) new_x,
st_y(st_transform(geo, 26918)) new_y
from testgeo
where name='point 2'
```

Real world data

- resources <https://postgis.net/workshops/postgis-intro/geometries.html>
- need to load data in postgis from ny data. copy the file nyc_data.backup to C:/pgadmin/data
- in power shell run this command if you using docker
- `docker exec -it postgis16 ls -lah /data/data`
- if we saw the nyc_data that means we can restore it by running this command:

```
docker exec -i your_container_name \
pg_restore -U postgres -d mynewdb < backup.dump
```

- `docker exec -it postgis16 pg_restore -U postgres -d nydat /data/data/nyc_data.backup`

- if we refresh the app db data base we will see new table ** make sure that postgis extension is already activated on the db that you will restore to it. otherwise you will get an error!
- check the data you have in the db, geometry type , by running :
- `select * from geometry_columns`

check columns and type for certain table :

`SELECT`

-
- `select column_name, data_typr,udt_name from information_schema.columns where table_name='nyc_cencus_blocks`
- if you table shows geometry (in one of the above) that means it uses geometry
- `SELECT * FROM geography_columns -- this is to check if we have any geography`

--simple sql statements

```
-- give me first 10 cencus block
select * from nyc_census_blocks
limit 10
```

```
-- give me the count of streets
select count(*)
from nyc_streets
```

```
-- count of streets that have name starts with a
select count(*)
from nyc_streets
where name like 'A%'
```

```
-- get the population based on the census blocks
select sum(popn_total)
from nyc_census_blocks
```

```
-- get pop for each boroname using group by
select sum(popn_total) as pop,boroname as name
from nyc_census_blocks
group by boroname
```

```
-- what is the most populated boroname ?
select sum(popn_total) as pop,boroname as name
from nyc_census_blocks
group by boroname
```

```
order by pop desc
limit 1
```

```
--
```

```
--find % race in each boro
```

```
select boroname,
100.0* (sum(popn_white)/sum(popn_total)) as white_prec,
100.0* (sum(popn_black)/sum(popn_total)) as black_prec,
100.0* (sum(popn_asian)/sum(popn_total)) as asian_prec,
100.0* (sum(popn_other)/sum(popn_total)) as other_prec,
100.0* (sum(popn_nativ)/sum(popn_total)) as native_prec
from nyc_census_blocks
group by boroname
```

-- nested select , select boros where is asian pop is higher that black pop other pop filed is more than 10% ?

```
select * from(
select boroname,
100.0* (sum(popn_white)/sum(popn_total)) as white_prec,
100.0* (sum(popn_black)/sum(popn_total)) as black_prec,
100.0* (sum(popn_asian)/sum(popn_total)) as asian_prec,
100.0* (sum(popn_other)/sum(popn_total)) as other_prec,
100.0* (sum(popn_nativ)/sum(popn_total)) as native_prec
from nyc_census_blocks
group by boroname) t
where asian_prec>black_prec or other_prec >10.0
```

-- having statement ! lets get the count of crimes committed by knife in each boro

```
select boroname,count(*)
from nyc_homicides
where weapon like '%knife%'
group by boroname
having count(*) >100
```

Now lets spatially explore the dataset !

```
select * from nyc_neighborhoods
```

```
-- find area of west village
select ST_area(geom) from nyc_neighborhoods
where name ='West Village'
```

```
-- what is the largest neighborhood in area
select name,st_area(geom) from nyc_neighborhoods
order by st_area(geom) DESC
limit 1
```

```
--what is the longest 3 streets in nyc
```

```
select name,st_length(geom) from nyc_streets
order by st_length(geom) desc
limit 3
```

```
-- what is the most north train station
select name,ST_astext(geom), ST_Y(geom) as y , ST_x(geom) as x from
nyc_subway_stations
order by ST_Y(geom) desc
limit 1
```

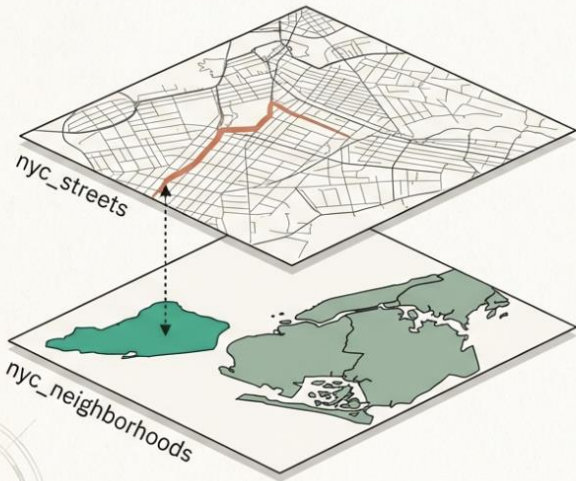
```
-- what is the total length of nyc streets
select sum(st_length(geom)) from nyc_streets
```

```
-- by type total length of nyc streets
select type ,sum(st_length(geom)) as totlenght from nyc_streets
group by type
order by totlenght
```


- 🗺️ What is a spatial join?

Joining on *Where*, Not *What*

Standard SQL joins match IDs (id = id). Spatial Joins match location.

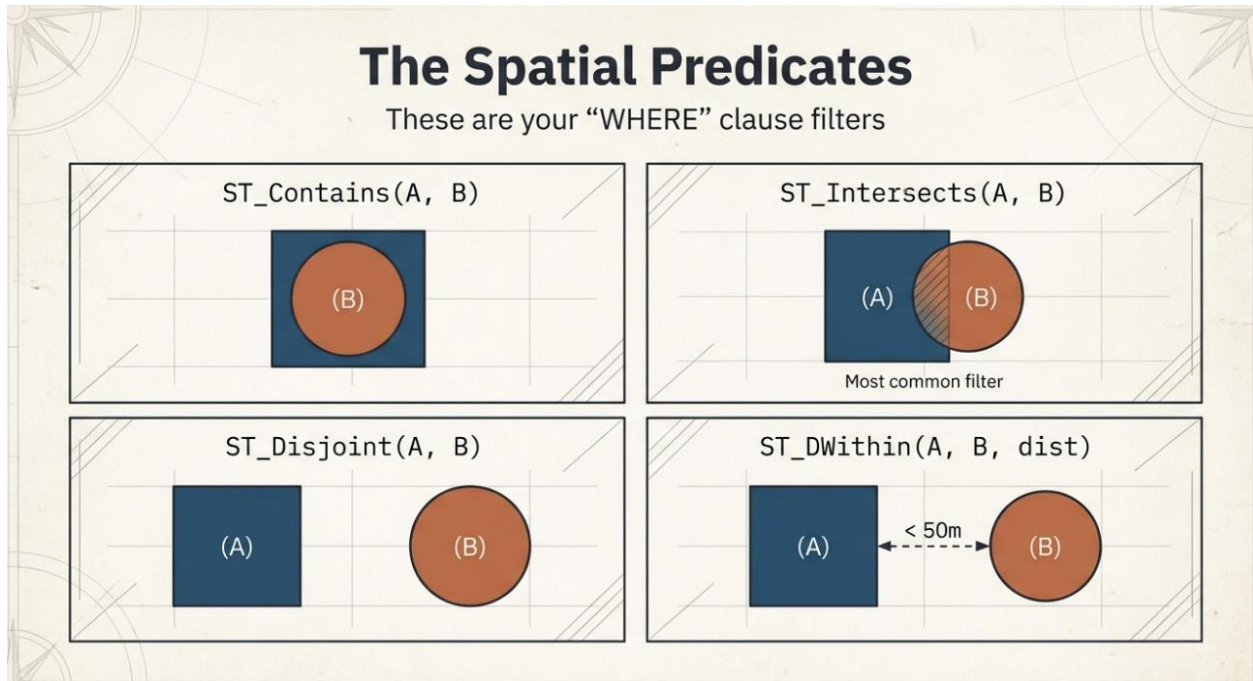


```

FROM nyc_streets s
JOIN nyc_neighborhoods n
  ON ST_Intersects(s.geom, n.geom)
WHERE n.borname = 'Staten Island'
```

-
- What is a JOIN? (plain English)
- A JOIN combines rows from two (or more) tables based on a relationship between them.
- FROM table_a
- JOIN table_b ON <condition>
- The condition decides which rows match.
- The type of JOIN decides which unmatched rows are kept or dropped.
 - Only matching rows INNER
 - Keep left even if missing LEFT
 - Keep right even if missing RIGHT
 - Keep everything FULL
 - All combinations CROSS
 - LATERAL JOIN (Proximity / Best-Match Join) For each row on the left, run a spatial search on the right and return the best match.
 - Hierarchy in same table SELF
- 🗺️ What is a spatial join?
- A spatial join joins rows from two tables based on where geometries are, not on IDs.
- FROM table_a a

- `JOIN table_b b`
- `ON <spatial predicate>`
- The predicate decides the spatial relationship.



- - streets in staten island

```
select * from nyc_streets s
join nyc_neighborhoods n
on st_intersects(s.geom,n.geom)
where n.boriname='Staten Island'
```

- -- give me the names of all subway stations in Manhattan

```

select t.name,ST_astext(t.geom),n.boriname from nyc_neighborhoods
n

join nyc_subway_stations t

on st_intersects(n.geom,t.geom)

where n.boriname ='Manhattan'

```

- -- how many crime within 200 for each subway

```

select t.name,count(*) as ccount

from nyc_subway_stations t

join nyc_homicides h

on st_dwithin(t.geom,h.geom,200)

group by t.name

order by ccount desc

```

- --Rank neighborhoods by length of street inside them

```

select

n.boriname,

sum(st_length(ST_CollectionExtract(st_intersection(s.geom,n.geom)
,2)))as segmentlen

from nyc_neighborhoods n

join nyc_streets s

on st_intersects(n.geom,s.geom)

group by n.boriname

order by segmentlen desc

```

-
- -- using st distance, slow for big data set , DISTINCT ON keeps the first row per group,
- --based on the ORDER BY.
-
- - question: what is the nearest street for each crime ?

```
SELECT DISTINCT ON (c.id)

    c.id, s.name

FROM nyc_homicides c

JOIN nyc_streets s

    ON ST_DWithin(c.geom, s.geom, 50)

ORDER BY c.id, ST_Distance(c.geom, s.geom);
```

The join is limited by ST_DWithin

postgres will (usually) use the GiST index on nyc_streets.geom to find only streets within 50 units of each homicide point.

So it's not point × all streets.

But ST_Distance is computed for all candidate pairs that pass ST_DWithin

For each homicide, if there are N streets within 50, then Postgres computes ST_Distance N times for that homicide (so total = sum of candidates across all homicides).

DISTINCT ON (c.id) then picks the first row per homicide after sorting

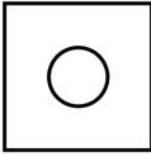
That means it must order the candidate streets for each c.id by distance (or do an equivalent plan).

- **Spatial relationships /compare relationships between geometries**
- **Function Key idea**
- ST_Contains Strict inside
- ST_Within Reverse of contains
- ST_Intersects Any overlap
- ST_Disjoint No overlap
- ST_Touches Boundary only
- ST_Overlaps Partial overlap

- ST_Equals Exactly same
- ST_Crosses Cross through
- ST_Distance How far
- ST_DWithin Within distance

ST_Contains — Strict inside

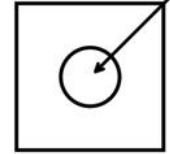
A contains B



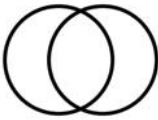
A = square, B = circle

ST_Within — Reverse of contains

B within A



ST_Intersects — Any overlap



They overlap at least a little

ST_Disjoint — No overlap



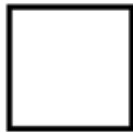
No shared space

ST_Touches — Boundary only



Meet only at the boundary

ST_Equals — Exactly same



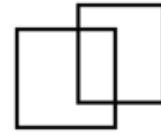
Same shape, same location

ST_Distance — How far



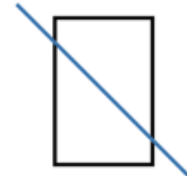
Returns a number

ST_Overlaps — Partial overlap



Shared area, but neither fully contains the other

ST_Crosses — Cross through



Line passes through polygon

ST_DWithin — Within distance



True if distance <= given value

--create example data and test the functions A square polygon +
points + lines

Make sure to remove the "--" from the front of the functions to test
them

WITH

poly AS (

SELECT ST_GeomFromText(

'POLYGON((0 0, 10 0, 10 10, 0 10, 0 0))',

4326

) AS geom

),

```

pt_inside AS (
    SELECT ST_GeomFromText('POINT(5 5)', 4326) AS geom
),

pt_edge AS (
    SELECT ST_GeomFromText('POINT(0 5)', 4326) AS geom
),

pt_outside AS (
    SELECT ST_GeomFromText('POINT(20 20)', 4326) AS geom
),

-- LINE that touches the polygon boundary only
line_touches AS (
    SELECT ST_GeomFromText(
        'LINESTRING(-5 5, 0 5)',
        4326
    ) AS geom
),

-- LINE that crosses through the polygon interior
line_crosses AS (
    SELECT ST_GeomFromText(
        'LINESTRING(-5 5, 15 5)',
        4326

```

```

    ) AS geom
),

-- LINE completely outside (no touch, no cross)
line_disjoint AS (
    SELECT ST_GeomFromText(
        'LINESTRING(20 20, 30 30)',
        4326
    ) AS geom
)

-- point inside >> true
--select st_intersects(pt_inside.geom,poly.geom)
--from poly, pt_inside;

-- outside point >> should be false
--select st_intersects(pt_outside.geom,poly.geom)
--from poly, pt_outside;

-- Line crosses >> true
--select st_intersects(line_crosses.geom,poly.geom)
--from poly, line_crosses;

```



```

--disjoint >> ture

--select ST_Disjoint(line_disjoint.geom,poly.geom)

--from poly, line_disjoint;


--ST_Contains , poygon contains the point , if you swich it it will
give you false

--select ST_Contains(pt_inside.geom,poly.geom)

--from poly, pt_inside;


-- point on the edge is not contained !

--select ST_Contains(poly.geom,pt_edge.geom)

--from poly, pt_edge;


-- ST_Within      is the opposite of contains

--select ST_Within(pt_inside.geom,poly.geom)

--from poly, pt_inside;


--ST_Touches

--select ST_Touches(pt_inside.geom,poly.geom)

--from poly, pt_inside;


-- if the point touches the boundary only

--select ST_Touches(pt_edge.geom,poly.geom)

--from poly, pt_edge;

```

```
-- point touches

--select ST_Touches(line_touches.geom,poly.geom)

--from poly, line_touches;


-- use ST_Crosses

--select st_crosses (line_crosses.geom,poly.geom)

--from poly,line_crosses


--ST_Distance

--The ST_Distance(geometry A, geometry B) calculates the shortest
--distance between two geometries and returns it as a float.

--This is useful for actually reporting back the distance between
objects.


--select st_Distance(pt_inside.geom,pt_outside.geom)

--from pt_inside,pt_outside
```

union function

```
select st_union(public.nyc_census_blocks.geom)
```

- from public.nyc_census_blocks
- lets try it on the nyc_neighborhoods

```
SELECT gid, ST_IsValidReason(geom) AS why  
FROM nyc_neighborhoods  
WHERE NOT ST_IsValid(geom);
```

-- we can make it valid and run it

- select st_union(a.geom) from (select st_makevalid(geom) as geo from nyc_neighborhoods) as a
- -- ST_CollectionExtract(geometry, type)
- This function becomes crucial after ST_MakeValid, ST_Intersection, or ST_Union, because those often return mixed geometry collections.
- Geometry type codes
- Code Geometry
- 1 POINT
- 2 LINESTRING
- 3 POLYGON

```
with newtable as (  
select ST_CollectionExtract(st_makevalid(geom),3) as geom  
from public.nyc_neighborhoods  
)
```

- select st_union(newtable.geom) from newtable

Spatial indexes

What is an index?

An index is like the **index of a book**.

- Without an index → you read **every page**
- With an index → you jump **straight to the answer**

Why indexes exist

Indexes make queries **fast**.

Without an index:

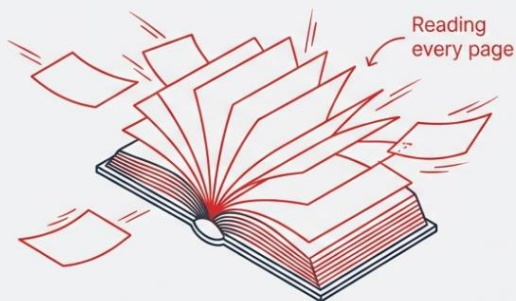
Check every row (slow)

With an index:

Go directly to matching rows (fast)

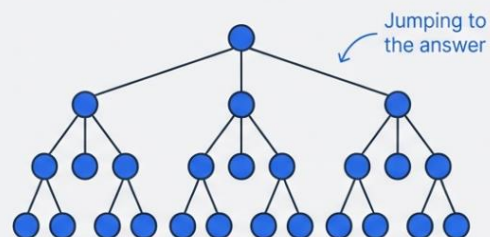
The Need for Speed: Spatial Indexing

The Problem: Full Table Scan



Without an index, the database scans every row sequentially. Slow.

The Solution: Indexing



B-Tree (Standard)	GiST (Spatial)
Ordered data (IDs, Names, Dates). 1, 2, 3...	Spatial proximity. Indexes Bounding Boxes, not geometry.

```
CREATE INDEX idx_geom ON table USING GIST (geom);
```

PostgreSQL index types

1 B-Tree (MOST COMMON)

- Default index type
- Used for:
 - IDs /names/dates/numbers

```
CREATE INDEX idx_name ON table(column);
```

Spatial indexing is one of the three core features of a spatial database (along with geometry types and spatial functions).

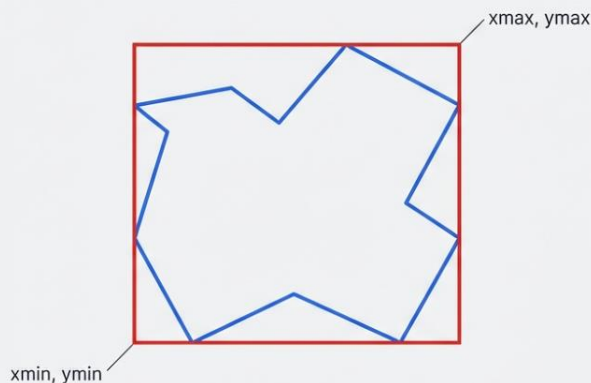
2 GiST (Spatial index)

- Used for **PostGIS geometries**
- Indexes **bounding boxes** **GiST indexes bounding boxes, not geometries.**
- **Each geometry becomes:**

xmin, ymin, xmax, ymax

- Required for: maps/distance queries/spatial joins

How GiST Works: The Bounding Box



Spatial Join (10k x 10k rows):

Without Index: 100,000,000 comparisons (N x M)

With Index: ~20,000 comparisons (N x constant)



Critical: ST_Distance does NOT use indexes. Use ST_DWithin for proximity searches.

```
CREATE INDEX idx_geom ON table USING GIST (geom);
```

The tree is organized by spatial proximity, not value order.

World

├── Region A

| ├── Box 1

| ├── Box 2

├── Region B

| ├── Box 3

| └── Box 4

3 BRIN (Big data, simple patterns)

- Very small index in size , Instead of indexing **every row**, BRIN indexes **ranges of table blocks**.
- Used when data is naturally ordered append only data
- Example: GPS logs, timestamps, Auto-increment IDs

SP-GiST

- Optimized for point data
- Faster for large point-only datasets
- Limited use for lines/polygons

Cost of Spatial Joins (Why Indexes Matter)

Consider joining two spatial tables:

- Table A: 10,000 geometries

- Table B: 10,000 geometries

Without an index:

$10,000 \times 10,000 = 100,000,000$ comparisons

With a spatial index:

Compare only geometries whose bounding boxes overlap,

$\approx 20,000$ comparisons this is an estimate , but if you have organized data well distributed this number is good estimate , why , example streets is connected to 2 other streets

Scenario	Comparisons
No index	$N \times M$
With spatial index	$N \times \text{small_constant}$

What indexes speed up

- ✓ WHERE
 - ✓ JOIN
 - ✓ ORDER BY (sometimes)
-

What indexes do NOT speed up

- ✗ `SELECT *`
 - ✗ Functions that don't use indexes
 - ✗ Very small tables
-

One thing to remember

Indexes make **reading fast**, but make **writing slower**
(INSERT / UPDATE / DELETE)

Spatial Functions That Use Indexes

Only some functions trigger spatial index usage automatically.

Most important indexed functions:

- `ST_Intersects`
- `ST_Contains`
- `ST_Within`
- `ST_DWithin`
- `ST_Covers`
- `ST_CoveredBy`

Critical note:

`ST_Distance` **does not** use indexes for filtering.

```
create index idx_nyc_census_blocks
on nyc_census_blocks
using gist(geom)
```

```
create index idx_nyc_neighborhoods  
on nyc_neighborhoods  
using gist(geom)
```

KNN search

```
SELECT *  
  
FROM public.nyc_homicides  
  
ORDER BY geom <-> ST_SetSRID(ST_Point(592160,  
4502211),26918 )  
  
LIMIT 5;
```

```
SELECT *  
  
FROM public.nyc_homicides  
  
where st_dwithin(geom,ST_SetSRID(ST_Point(592160,  
4502211),26918 ),200)  
  
order by st_distance(geom,ST_SetSRID(ST_Point(592160,  
4502211),26918))
```

- When you write:

```
ORDER BY c.geom <-> s.geom
```

Postgres recognizes this as a K-Nearest Neighbor (KNN) query.

What Postgres does internally

Uses the GiST index on s.geom , Traverses the index tree , Finds geometries closest first. Stops as soon as it finds the top result (with LIMIT 1)

- Why we ALSO use `ST_DWithin`

```
WHERE ST_DWithin(c.geom, s.geom, 50)
```

```
ORDER BY c.geom <-> s.geom
```

Because `<->`: Finds the nearest street even if it's very far away, Does not enforce a maximum distance. `ST_DWithin`: Enforces a hard cutoff (e.g., 50 meters/feet) and Prevents bad matches

- 📌 **Together they mean: “Among streets within 50 units, pick the closest one.”**
- what is the difference between left join and lateral join that is used in knn search ?
- [1] `LEFT JOIN` — a row-preservation rule

```
FROM crimes c
```

```
LEFT JOIN streets s
```

```
ON ST_Intersects(c.geom, s.geom)
```

- What `LEFT JOIN` does, Keeps all rows from the left table, Adds NULLs when no match exists on the right. Does not change how the right table is computed
- Think of it as : “Don’t drop crimes even if no street matches.”
- What it does not do , ❌ It does not: Run per-row logic ,Choose the “best” match ,Limit matches to one row
- Enable nearest-neighbor logic
- [2] `LATERAL` — a row-by-row execution rule

```
FROM crimes c
```

```
JOIN LATERAL (
```

```
SELECT *
```

```
FROM streets s
```

```
WHERE ST_DWithin(c.geom, s.geom, 50)
```

```
ORDER BY c.geom <-> s.geom
```

```
LIMIT 1) s ON true
```

- What `LATERAL` does
- Runs the subquery once per row of the left table
- Allows the subquery to reference left-side columns
- Enables “best match” logic (`ORDER BY ... LIMIT 1`)
- Think of it as : “For each crime, compute a custom result.”
- What it does not do
- ❌ It does not: Preserve unmatched rows (by itself), Replace `INNER` vs `LEFT` semantics

```

SELECT streets.gid, streets.name,

       ST_Transform(streets.geom, 4326),

       streets.geom <-> 'SRID=26918;POINT(583571.9
4506714.3) '::geometry AS dist

FROM

nyc_streets streets

ORDER BY dist

LIMIT 3;

```

K-Nearest Neighbors (KNN): The Concept

The Logic: Sort by distance, take the top 1.

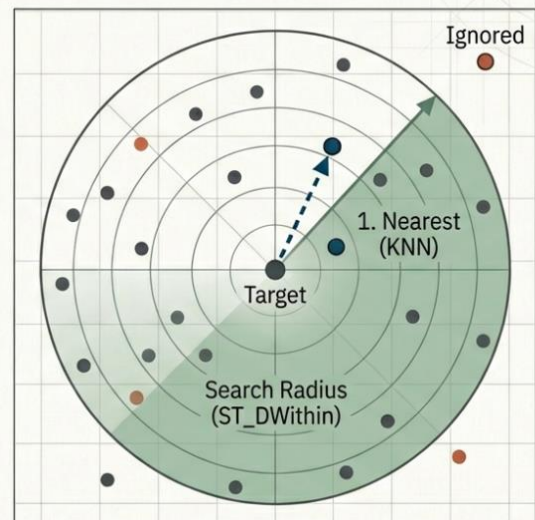
Operator: <-> (Distance Operator)

- **Constraint:** Pure KNN finds the nearest object even if it's 1,000 miles away.
- **Optimization:** Combine with ST_DWithin to set a maximum search radius.

```

ORDER BY geom <-> target_geom
LIMIT 1

```

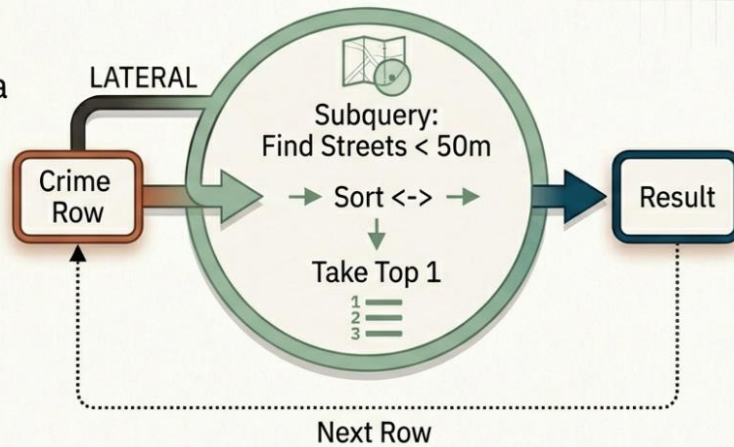


The Power of LATERAL Joins

Iterating row-by-row for nearest neighbor searches.

Problem: Standard LEFT JOIN can't efficiently run a "nearest" sort-and-limit for every row.

Solution: JOIN LATERAL triggers a subquery for each individual row on the left table.



- -- what is the nearest street to each crime in

```
select c.id,c.weapon,c.incident_d,near_street.name,c.geom
from public.nyc_homicides c
join lateral (
select s.name
from public.nyc_streets s
where st_dwithin(s.geom,c.geom,50)
order by c.geom <-> s.geom -- KNN nearest (uses GiST)
limit 1) near_street on true
```

- -- If you want to KEEP homicides even if no street is within 50 units use left join lateral
- -- 🔄 Next step: total homicides per street

- -- to make is easy use with as
-

```
with mynewcrimepoints as
```

```
(
```

```
select c.id,c.weapon,c.incident_d,near_street.name,c.geom
```

```
from public.nyc_homicides c
```

```
join lateral (
```

```
select s.name
```

```
from public.nyc_streets s
```

```
where st_dwithin(s.geom,c.geom,50)
```

```
order by c.geom <-> s.geom -- KNN nearest (uses GiST)
```

```
limit 1
```

```
) near_street on true
```

```
)
```

```
select count (*) numberofcrimes,mynewcrimepoints.name
```

```
from mynewcrimepoints
```

```
group by mynewcrimepoints.name
```

```
order by numberofcrimes desc
```

ANALYZE and VACUUM matter (often ignored)

ANALYZE

- Updates planner statistics
- Helps PostgreSQL decide:
 - index scan vs sequential scan
 - join order

Run after:

- Bulk loads
- Large deletes
- Major updates

```
SELECT *  
  
FROM pg_stat_user_tables  
  
where relname='nyc_streets'
```

```
ANALYZE table_name;
```

VACUUM

- Reclaims dead rows
- Keeps indexes efficient

Run after:

- Heavy updates/deletes

```
VACUUM ANALYZE table_name;
```

Autovacuum helps, but **manual VACUUM is still important** for spatial workloads.

This does two things:

1. **VACUUM**

- cleans dead rows

2. **ANALYZE**

- updates statistics

What Happens With DELETE

If you run:

```
DELETE FROM users WHERE id = 1;
```

The row is **not immediately removed**.

It becomes a **dead row** waiting for cleanup.

Why PostgreSQL Does This

This allows:

- **multiple transactions**
- **consistent reads**
- **no locking conflicts**

Example:

One user updating data while another is reading it.

The Problem With Dead Rows

If they accumulate:

- tables become **larger**
- queries become **slower**
- indexes grow unnecessarily

Index-Only Spatial Queries

Uses only the index , No exact geometry calculation , Very fast, but approximate

```
SELECT Sum(popn_total)
FROM nyc_neighborhoods neighborhoods
JOIN nyc_census_blocks blocks
ON neighborhoods.geom && blocks.geom
WHERE neighborhoods.name = 'West Village';
```

```
SELECT Sum(popn_total)
FROM nyc_neighborhoods neighborhoods
JOIN nyc_census_blocks blocks
ON ST_Intersects(neighborhoods.geom, blocks.geom)
WHERE neighborhoods.name = 'West Village';
```

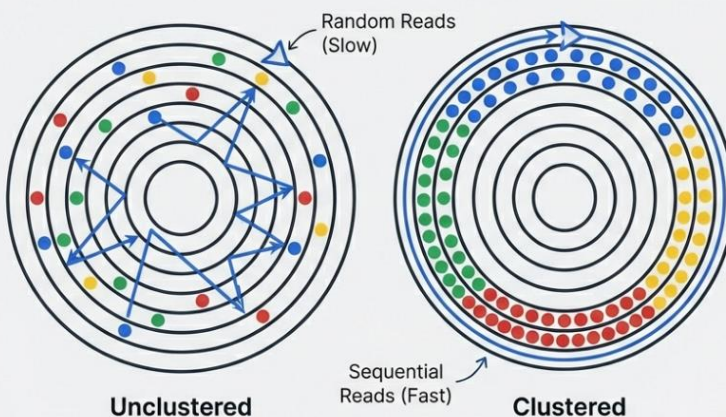
PostGIS – Clustering on Indices

◆ Big Idea

Databases retrieve data only as fast as they can read it from storage. Even with SSDs and RAM caching, **physical data layout still affects performance**.

Clustering = physically reordering table rows so records likely accessed together are stored close together on disk (and in memory).

Clustering: Aligning Physical Reality with Logic



Problem: Indexes find data fast, but disk physics is slow.

Solution: CLUSTER rewrites the table to physically store nearby geometries together.

```
CLUSTER nyc_census_blocks  
USING nyc_census_block_idx_geom;
```

 **Caveat:** Not permanent. New inserts ignore cluster order. Requires maintenance.

1 Why Clustering Matters

- Data is written to disk in no guaranteed order.
- Query patterns often follow logical relationships (e.g., spatial proximity).
- If related records are stored near each other:
 - Fewer disk reads

- Better cache efficiency
- Faster queries

👉 Indexes provide a *natural ordering* that matches how data is retrieved.

2 Clustering Based on an Index

When you cluster a table using an index:

- The database rewrites table rows in index order.
- Rows accessed together become physically closer.

Command:

```
CLUSTER table_name USING index_name;
```

3 Clustering on a Spatial (R-Tree) Index

Spatial data is usually queried by geographic windows (map views).

Since nearby objects are often retrieved together:

- Clustering spatial data improves performance.

Example:

```
CLUSTER nyc_census_blocks USING nyc_census_blocks_geom_idx;
```

This:

- Reorders table rows
- Matches physical storage to spatial proximity
- Speeds up spatial queries

⚠ Speed improvement may be small if data was already spatially ordered.

✓ What **CLUSTER** Actually Does

When you run:

```
CLUSTER table_name USING index_name;
```

PostgreSQL:

1. Reads the table
2. Rewrites it in the order of the index
3. Replaces the original table data with the reordered version

👉 Same table , No new table appears

👉 Same name , Same schema .

👉 Index remains unchanged , Table is locked during clustering

👉 Just different physical row order

Important

Clustering is **not permanent**:

- New inserts will not follow clustered order
- You must run `CLUSTER` again if needed

How `CLUSTER` Affects Indexes

1 Index is NOT rebuilt

When you run:

```
CLUSTER table_name USING index_name;
```

- ✓ PostgreSQL **does not recreate the index**
- ✓ It uses the index **only to determine row order**
- ✓ After clustering, indexes still work normally

👉 The index becomes more efficient because table rows now follow its order.

2 Other indexes still work

If your table has multiple indexes:

- All remain intact
- None are deleted
- Query planner can still use any of them

But:

- ✓ The clustering index benefits the most
- ✓ Others see little or no improvement

✓ How **CLUSTER** Affects **VACUUM**

◆ 1 **CLUSTER** already rewrites the table

Since **CLUSTER** rewrites the table:

- Dead tuples are removed
- Space is compacted
- Similar to:

VACUUM FULL

👉 Running **VACUUM FULL** right after clustering is unnecessary.

◆ 2 **Regular VACUUM** still needed

After clustering:

- Inserts and updates create fragmentation again
- Regular **VACUUM** remains important

CLUSTER is **not a replacement** for **VACUUM**.

Example how gist idx works with the cluster:

```
CREATE INDEX idx_geom ON table USING GIST (geom);
```

Organizes data using bounding boxes, Groups nearby geometries together

◆ 2 What Happens During CLUSTER on GiST

When you run:

```
CLUSTER table_name USING idx_geom;
```

PostgreSQL:

Reads the GiST index order

Rewrites table rows to follow that spatial grouping

Stores nearby geometries close together

◆ 3 Why This Improves Spatial Performance

Spatial queries usually retrieve nearby features:

- Map window searches
- Radius searches
- Intersection queries

Clustering reduces:

- ✓ Random disk reads
- ✓ Cache misses
- ✓ Query execution time

EXAMPLE

```
-- create index  
create index nyc_census_block_idx_geom  
on nyc_census_blocks using gist(geom)  
  
-- check performance  
EXPLAIN ANALYZE SELECT * FROM nyc_census_blocks;  
  
-- use cluster  
cluster nyc_census_blocks using nyc_census_block_idx_geom  
  
-- re check performance  
EXPLAIN ANALYZE SELECT * FROM nyc_census_blocks;
```

Quick recap .

ST_Centroid / ST_PointOnSurface / ST_Buffer / st_union / ST_NumGeometries

“What percentage of each neighborhood’s population lives within 500 meters of a subway station — without double counting?”

1 census blocks as missing the neighborhood information , so we need to join the blocks with the neighborhood , but the challenge here is to make sure that we dont double count the pop since one census might touch 2 or more neighborhoods , so we can use `st_centroid` to make sure that the center of the block falls inside the neighborhood and this is why we can assign the block to the neighborhood,

Challenge census block, we need to make sure that each subway station is matched with one census block to get the correct statistics.

The 2nd query (`ST_Contains(n.geom, ST_Centroid(b.geom))`) is usually *more accurate for population totals*, because it assigns **each census block to only one neighborhood**, so it avoids double counting.

Why the 1st query can be wrong

`ST_Intersects(n.geom, b.geom)` returns true if a block **touches** a neighborhood at *any* point (even a tiny sliver or just the border).

So if a block crosses or touches multiple neighborhood polygons, it gets counted **multiple times** → your `SUM(popn_total)` becomes **too high**.

Why the centroid method is usually better

`ST_Contains(n.geom, ST_Centroid(b.geom))` counts a block only if its **center point** is inside the neighborhood.

That makes it a **one-block** → **one-neighborhood** assignment (in practice), which is what you want when summing population.

```
select  sum(b.popn_total)
from nyc_neighborhoods n
join nyc_census_blocks b
on st_intersects (n.geom,b.geom)
```

```
select  sum(b.popn_total)
from nyc_neighborhoods n
join nyc_census_blocks b
on ST_Contains (n.geom,ST_Centroid(b.geom))
```

CODE

```
WITH blocks_with_neighborhood AS (
    SELECT
        b.blkid,
        b.popn_total,
        n.name AS neighborhood,
        b.geom as geom
    FROM nyc_census_blocks b
    JOIN nyc_neighborhoods n
    ON ST_Contains(n.geom, ST_Centroid(b.geom))
),
```

```
blocks_near_subway AS (
    SELECT DISTINCT ON (b.blkid)
        b.blkid,
        b.popn_total,
        b.neighborhood
    FROM blocks_with_neighborhood b
    JOIN nyc_subway_stations s
```

```
    ON ST_DWithin(b.geom, s.geom, 500)
),
neighborhood_total as (
select neighborhood,sum(popn_total) as total_pop
from blocks_with_neighborhood
group by neighborhood
),
subway_total as (
select neighborhood,sum(popn_total) as subway_total
from blocks_near_subway
group by neighborhood
)
select s.neighborhood,100*s.subway_total/n.total_pop as precentage
from neighborhood_total n
join subway_total s
on n.neighborhood=s.neighborhood
where total_pop>0
order by precentage desc
```

PostGIS Validity

Big idea

Most **TopologyException** errors happen because **one or more geometries are invalid**.

- **Points** → always valid
- **Lines** → always valid
- **Polygons / MultiPolygons** → can be invalid

Validity matters because **spatial algorithms assume clean structure**.
If structure is broken → wrong answers or errors.

1 What does “valid” mean?

A **valid polygon** must follow these rules (OGC standard):

1. Rings must **close**
2. Holes must be **inside** the outer ring
3. Rings **cannot self-intersect**
4. Rings may only touch at **points**, not edges
5. Polygons in a multipolygon **cannot touch each other**

📌 These rules exist so geometry math can work **fast and correctly**.

3 Detecting invalid geometry [ST_IsValid](#) / 🔍 Find the REASON
[ST_IsValidReason](#) / 4 Repairing invalid geometry ([ST_MakeValid](#))

The Rules of Geometry: Enforcing Validity

Garbage In, Garbage Out. Spatial algorithms require valid topology.



No Self-Intersections



Rings Must Close



Holes Must Be Inside



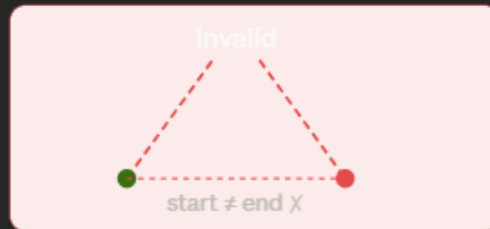
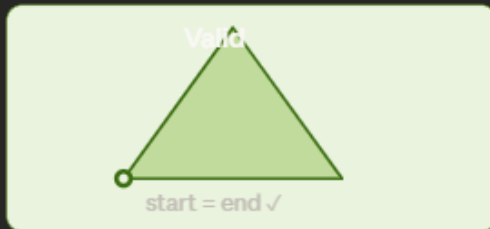
Valid Geometry

The Toolkit

<code>ST_IsValid(geom):</code>	Detects errors.
<code>ST_IsValidReason(geom):</code>	Diagnoses the specific rule violation.
<code>ST_MakeValid(geom):</code>	Attempts to repair the structure automatically.

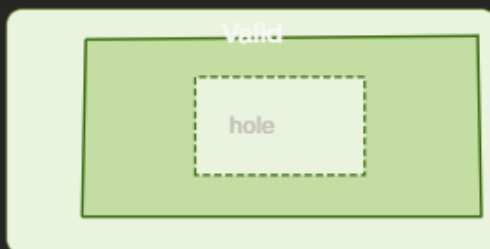
Rule 1 — Rings must close

The last point of a ring must equal the first point.



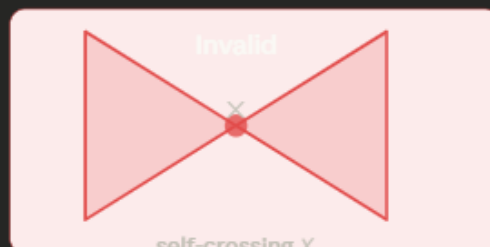
Rule 2 — Holes must be inside the outer ring

Inner rings (holes) must lie fully within the exterior boundary.



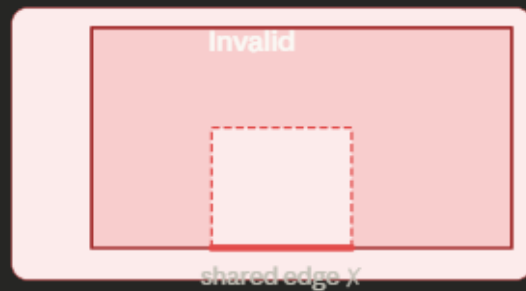
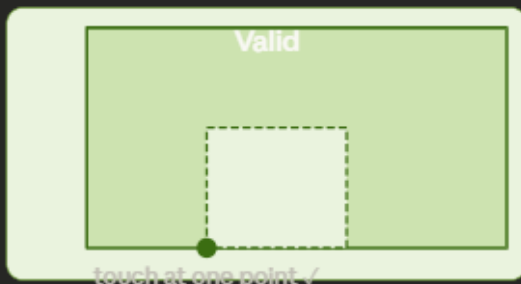
Rule 3 — Rings cannot self-intersect

No ring may cross over itself — the boundary must be simple.



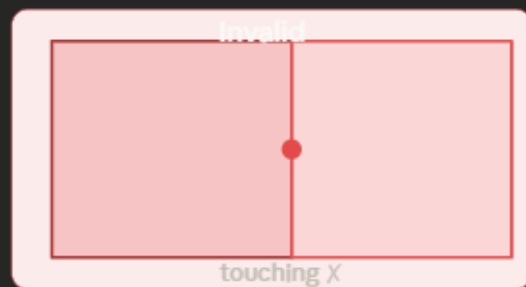
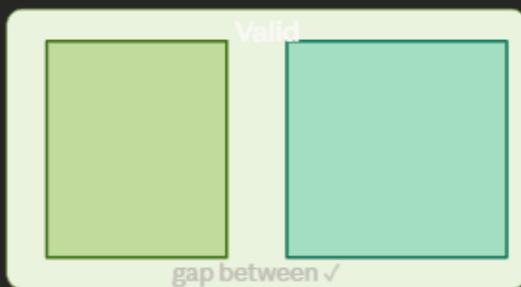
Rule 4 — Rings may only touch at points, not edges

The outer ring and a hole may share a single vertex, but never a shared edge.



Rule 5 — Polygons in a multipolygon cannot touch each other

Individual polygons within a multipolygon must be fully separate — no shared points.



Ploygoy ((1 1,2 2 , 3 3 ,1 1),(2 2 ,3 3 , 22)


```

-- list only invalid poly
SELECT NAME, boroname, geom
from nyc_neighborhoods
where not ST_isvalid(geom)

-- add reason ?
SELECT NAME, boroname, st_isvalidreason(geom) as why, geom
from nyc_neighborhoods
where not ST_isvalid(geom)

-- make it valid
SELECT NAME, boroname, geom, st_makevalid(geom)
from nyc_neighborhoods
where not ST_isvalid(geom)

-- fix the geometry in it in the table
update nyc_neighborhoods
set geom=st_makevalid(geom)
where not st_isvalid(geom)

```

ST_Equals

Check if there is any spatially duplicated geometry

```

SELECT a.name AS name_a, b.name AS name_b
FROM nyc_neighborhoods a
JOIN nyc_neighborhoods b
ON ST_Equals(a.geom, b.geom)

```

Or you can use this one

```
select * from (  
  SELECT a.name, b.name,  
         CASE WHEN ST_Equals(a.geom, b.geom)  
              THEN 'Spatially Equal'  
              ELSE 'Not Equal' END as s  
  FROM nyc_neighborhoods AS a, nyc_neighborhoods AS b  
) q  
where s='Spatially Equal'
```

Question , what are the streets start inside East Village AND end outside east village

```
select s.name,s.geom from nyc_streets s  
join nyc_neighborhoods n  
on st_contains(n.geom,st_startpoint(s.geom))  
where n.name ='East Village' and not  
st_contains(n.geom,st_endpoint(s.geom))
```

More Spatial Functions (St_Dump/ST_Union/ST_Start/End/Npoint)

New table by boroname using ST_Union

ST_Union = merges geometries into one

```
select st_union( ST_GeomFromText(
  'MULTILINESTRING((0 0, 5 5),(10 10, 20 20))' ))
```

```
CREATE TABLE nyc_neighborhoods_by_boron AS
SELECT
  boroname,
  ST_Union(geom) AS geom
FROM nyc_neighborhoods
GROUP BY boroname;
```

New streets table using ST_Dump

ST_Dump() = breaks a multi-geometry into individual single geometries

(ST_Dump(geom)).geom

(ST_Dump(geom)).path

1- geom → the geometry

2- path → position inside the original geometry

```
SELECT ST_Dump(
  ST_GeomFromText(
    'MULTILINESTRING((0 0, 5 5),(10 10, 20 20))' ))
```

You can use ST_LineMerge(geom) too to get one single line

```
CREATE TABLE nyc_streets_dump AS
SELECT
  s.name,
  (ST_Dump(s.geom)).geom AS geom
```

```
FROM nyc_streets s;
```

For East Village , what are the streets starts inside it and finishes outside

```
SELECT s.name, s.geom
FROM nyc_streets_dump s
JOIN nyc_neighborhoods_by_boron n
  ON n.boroname = 'Queens'
WHERE ST_Contains(n.geom, ST_StartPoint(s.geom))
      AND NOT ST_Contains(n.geom, ST_EndPoint(s.geom));
```

start inside + line intersects boundary + end outside

```
WHERE ST_Contains(n.geom, ST_StartPoint(s.geom))
      AND NOT ST_Contains(n.geom, ST_EndPoint(s.geom))
      AND ST_Intersects(s.geom, ST_Boundary(n.geom))
```

start inside + line intersects boundary + end outside

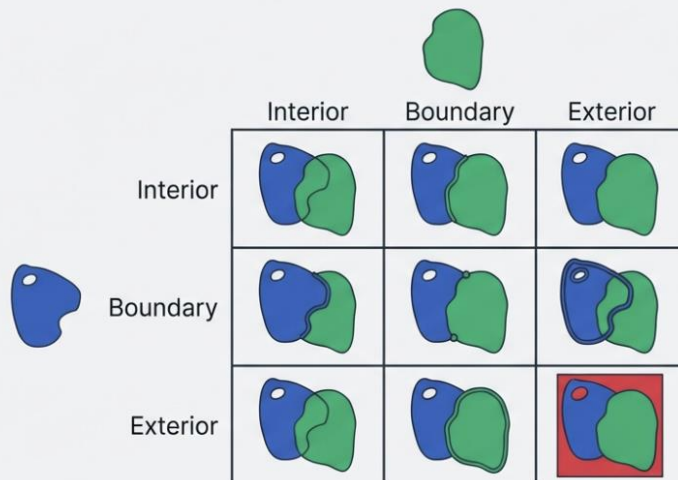
Ensures **exactly one** intersection point with the boundary

```
WHERE ST_Contains(p.geom, ST_StartPoint(l.geom))
      AND NOT ST_Contains(p.geom, ST_EndPoint(l.geom))
      AND ST_NPoints(ST_Intersection(l.geom, ST_Boundary(p.geom))) = 1
```

DE9IM (Dimensionally Extended 9-Intersection Model)

The Matrix: Understanding Relationships (DE9IM)

The Dimensionally Extended 9-Intersection Model analyzes how two shapes (A and B) interact based on their Interior, Boundary, and Exterior.



PostGIS uses this matrix to determine predicates:

- ST_Contains
- ST_Touches
- ST_Overlaps
- ST_Relate (Custom patterns)

ST_Relate in **PostGIS** is one of the most powerful (and confusing) spatial functions. It checks the exact spatial relationship between two geometries using a **DE-9IM matrix** (Dimensionally Extended 9-Intersection Model)

`ST_Relate(geomA, geomB)`

→ Returns a **9-character string** describing how the geometries relate.

OR (more useful form):

`ST_Relate(geomA, geomB, 'pattern')` pattern uses **T / F / ***

→ Returns **TRUE/FALSE** based on a pattern you define.

Example 1: Point inside Polygon

```
SELECT ST_Relate(  
  ST_GeomFromText('POINT(5 5)'),
```

```
ST_GeomFromText('POLYGON((0 0,10 0,10 10,0 10,0 0))')
);
```

Result:

0FFFFFF212

Check if the point With in

```
SELECT ST_Relate(
  ST_GeomFromText('POINT(5 5)'),
  ST_GeomFromText('POLYGON((0 0,10 0,10 10,0 10,0 0))'),
  'T**F***');
```

TRUE

Relationship	Pattern
Contains	T*****FF*
Within	T**F***F***
Touches	FT*****
Crosses	T*T*****
Disjoint	FF*FF****

T = must intersect

F = must NOT intersect

***** = don't care

Use **ST_Relate** when:

- You need full control
- Built-in functions aren't enough:
 - **ST_Contains**
 - **ST_Intersects**
 - **ST_Touches**

What DE9IM

- DE9IM describes the spatial relationship between ONE geometry A and ONE geometry B
- SQL applies the test pair-by-pair across datasets

2 Geometry Components (ALWAYS TRUE)

Every geometry has **three conceptual parts**:

Polygon

- Interior = area inside
- Boundary = ring(s)

- Exterior = everything else

Line

- Interior = the line except endpoints
- Boundary = endpoints
- Exterior = everything else

Point

- Interior = the point
- Boundary = empty
- Exterior = everything else

212FF1FF2

gem b

A

	Interior (b)	Boundary (b)	Exterior (b)
Interior (a)	$I \cap I$ Do they overlap in their interiors? core of "contains"	$I \cap B$ Does a's inside cross b's edge? a crosses into b	$I \cap E$ Does a extend outside b? a not fully inside b
Boundary (a)	$B \cap I$ Does a's edge enter b's inside? b crosses into a	$B \cap B$ Do their edges touch or share? core of "touches"	$B \cap E$ Does a's edge stick out of b? a not fully enclosed
Exterior (a)	$E \cap I$ Does b extend outside a? b not fully inside a	$E \cap B$ Does b's edge stick out of a? b not fully enclosed	$E \cap E$ Do they share any outside space? almost always true

	I (b) b's interior	B (b) b's boundary	E (b) b's exterior
I (a) a's interior	Overlap shared area?	Touch inside crosses edge?	Escape a spills out of b?
B (a) a's boundary	Boundary inside? a's edge dips in b?	Edges touch do edges meet?	Boundary exposed a's edge outside b?
E (a) a's exterior	Leftover I b escapes outside a?	Leftover B b's edge outside a?	Leftover outside shared outer space?

The “9 intersections” matrix

DE9IM checks the dimension of intersections between:

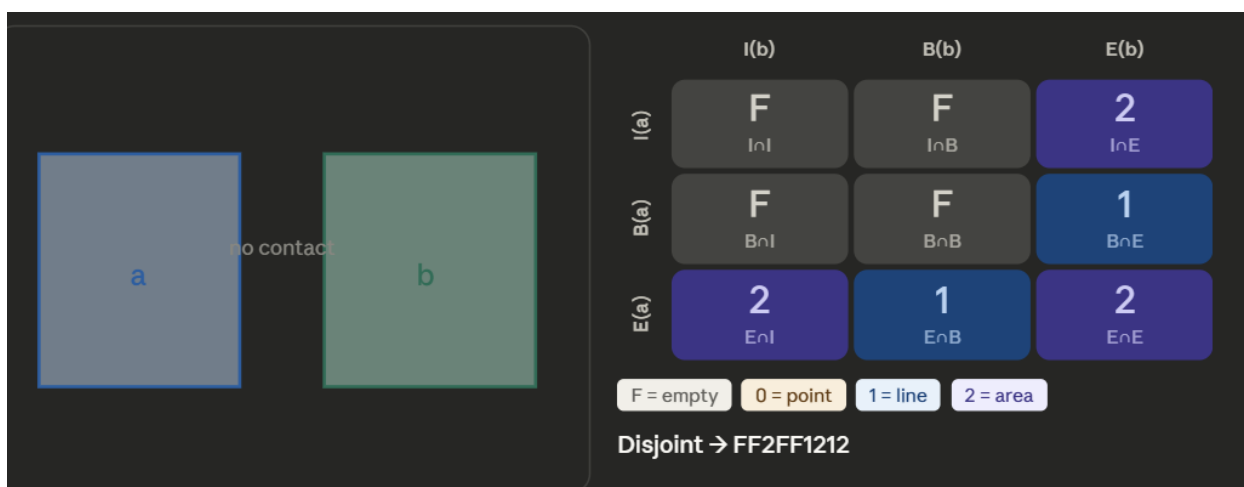
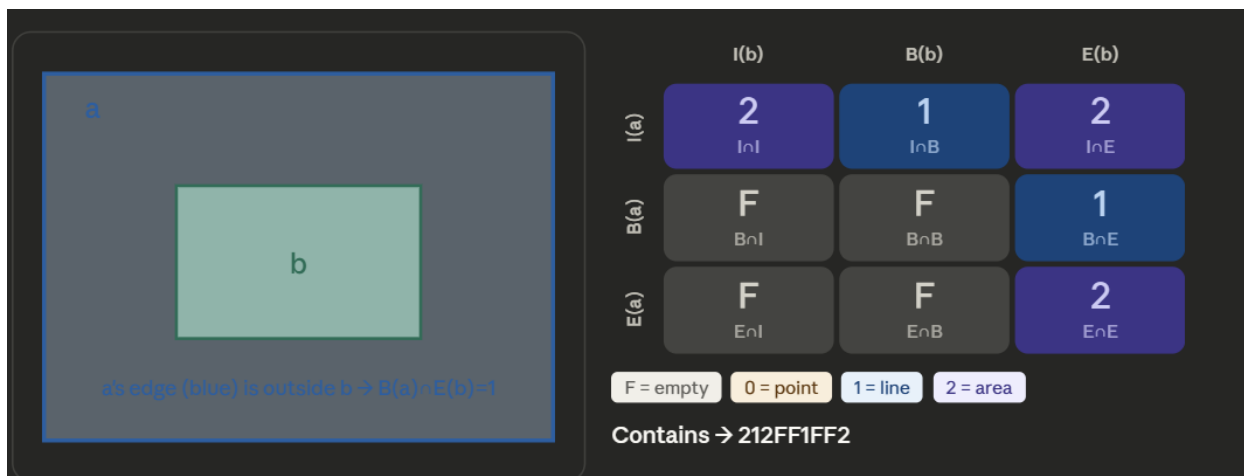
```

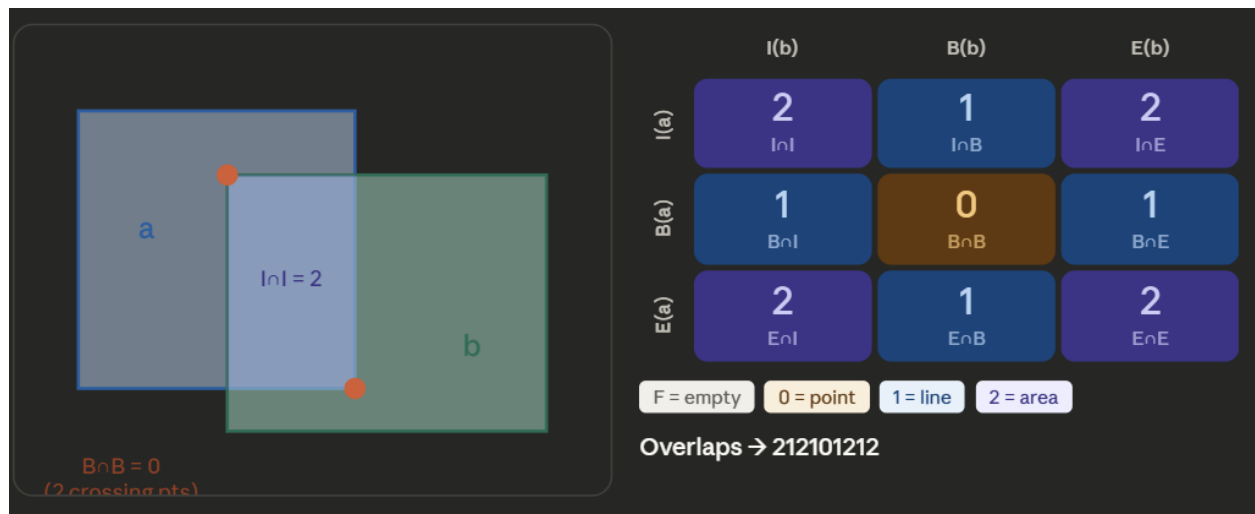
      B_geom
      I   B   E
A_geom I
      B
      E

```

Each cell stores:

- **2** = area intersection
- **1** = line intersection
- **0** = point intersection
- **F** = empty (no intersection)





PostGIS can compute the matrix using **ST_Relate**.

`ST_Relate(A.geom, B.geom)` -- returns 9-character DE9IM string

`ST_Relate(A.geom, B.geom, 'pattern')` -- returns boolean



	Interior	Boundary	Exterior
Interior	 $\dim(\dots) = 2$	 $\dim(\dots) = 1$	 $\dim(\dots) = 2$
Boundary	 $\dim(\dots) = 1$	 $\dim(\dots) = 0$	 $\dim(\dots) = 1$
Exterior	 $\dim(\dots) = 2$	 $\dim(\dots) = 1$	 $\dim(\dots) = 2$

★ RULES

I/I = overlap

I/B & B/I = touching

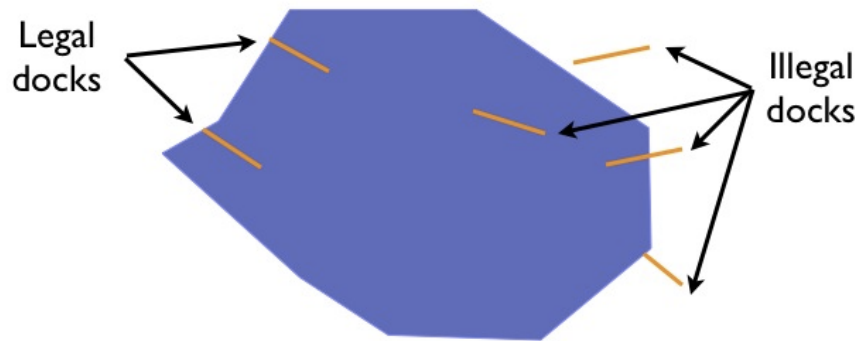
B/E & E/B = exposure

B/B = border contact

I/E = containment

E/* = what remains after removing the geometry

	I	B	E
I	Overlap	Touch	escape?
B	boundary inside?	Edges touch	boundary exposed?
E	Leftover I	Leftover B	Leftover Outside



Our legal docks have the following characteristics:

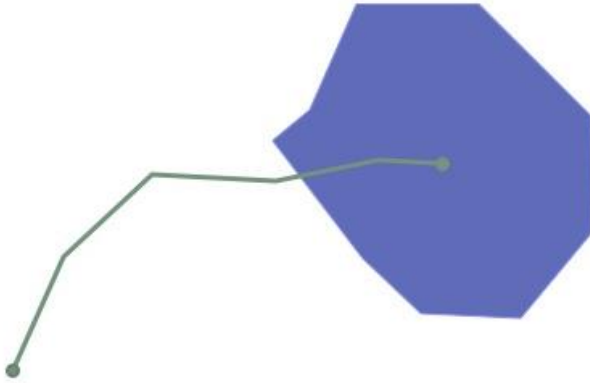
- Their interiors have a linear (1D) intersection with the lake interior
- Their boundaries have a point (0D) intersection with the lake interior
- Their boundaries **also** have a point (0D) intersection with the lake boundary
- Their interiors have no intersection (F) with the lake exterior

So their DE9IM matrix looks like this:



	<i>I</i>	<i>B</i>	<i>E</i>
<i>I</i>	I	F	F
<i>B</i>	0	0	F
<i>E</i>	2	1	2

Here's another example, of a linestring partially entering a polygon:



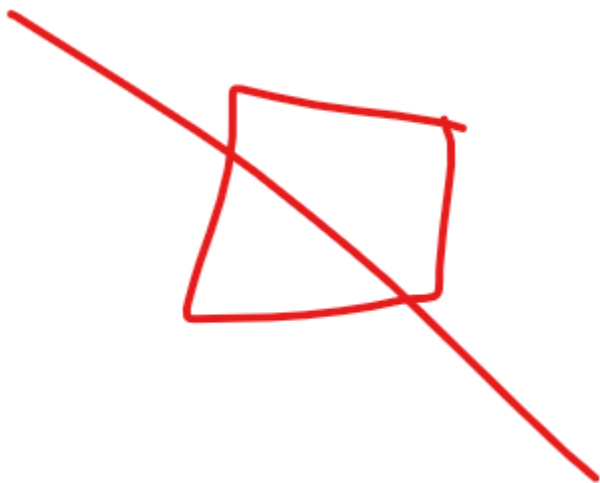
The DE9IM matrix for the interaction is this:



	<i>I</i>	<i>B</i>	<i>E</i>
<i>I</i>	1	0	1
<i>B</i>	0	F	0
<i>E</i>	2	1	2



	i	b	e
i	1	0	1
b	0	f	0
e	2	1	2



1	i	b	e
i	1	0	1
b	f	f	0
e	2	1	2

101FF0212

If you are unsure about the pattern use `ST_Relate(A.geom, B.geom)` it will give you the answer.

T, F, and * are for matching, not describing

When you already *know* what relation you want to test for, you don't need exact numbers — you just need to know which cells must have *something* (T), which must have *nothing* (F), and which you don't care about (*).

Example: T*****

This is a 9-character string representing the matrix.

T*****

Meaning:

- Only condition:
 - $\text{Interior}(A) \cap \text{Interior}(B) = T$
- Everything else $\rightarrow$ **don't care**

👉 This describes:

- Overlaps
 - Contains
 - Within
- (all cases where interiors intersect)

Example: Disjoint

Pattern:

FF*FF****

Meaning:

- Interior $\cap$ Interior = F
- Interior $\cap$ Boundary = F
- Boundary $\cap$ Interior = F
- Boundary $\cap$ Boundary = F

👉 No touching at all → completely separate

Example: Touches

Pattern:

FT*****

Meaning:

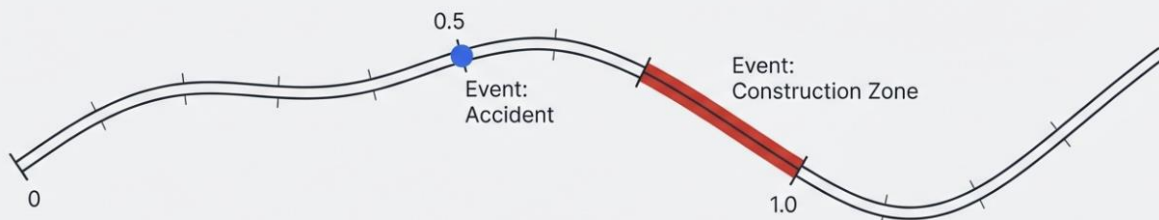
- Interior $\cap$ Interior = F ✗ (no overlap)
- Interior $\cap$ Boundary = T ✓ (they touch)

Everything else doesn't matter.

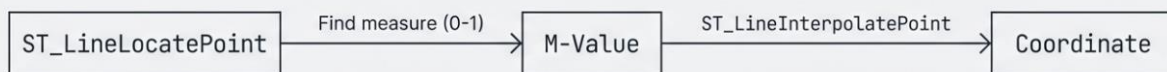
🔑 PostGIS — Linear Referencing

Linear Referencing: The 'M' Value

Dynamic Segmentation without splitting geometry



Geometry(Point, X, Y, Z, **M**)



◆ What is Linear Referencing?

Linear Referencing (LRS) = describing locations **along a line** using a **measure** instead of storing separate geometries.

Instead of saying "the accident happened at longitude 35.6, latitude 31.9", you say "the accident happened at kilometer 42 along Highway 6." That kilometer value is called the **measure (M)**. The line itself is the reference, and you locate everything by how far along it something is.

✅ Real-World Examples

- Highway mile markers
- Road maintenance segments
- River reach measurements

👉 Instead of storing full geometries, we store:

Road ID + distance along road

◆ Why Use Linear Referencing?

- ✓ No need to duplicate geometry
 - ✓ Events automatically adjust if base geometry changes
 - ✓ Saves storage
 - ✓ Easier updates
 - ✓ Ideal for networks (roads, rivers, rails)
-

◆ Base Concept

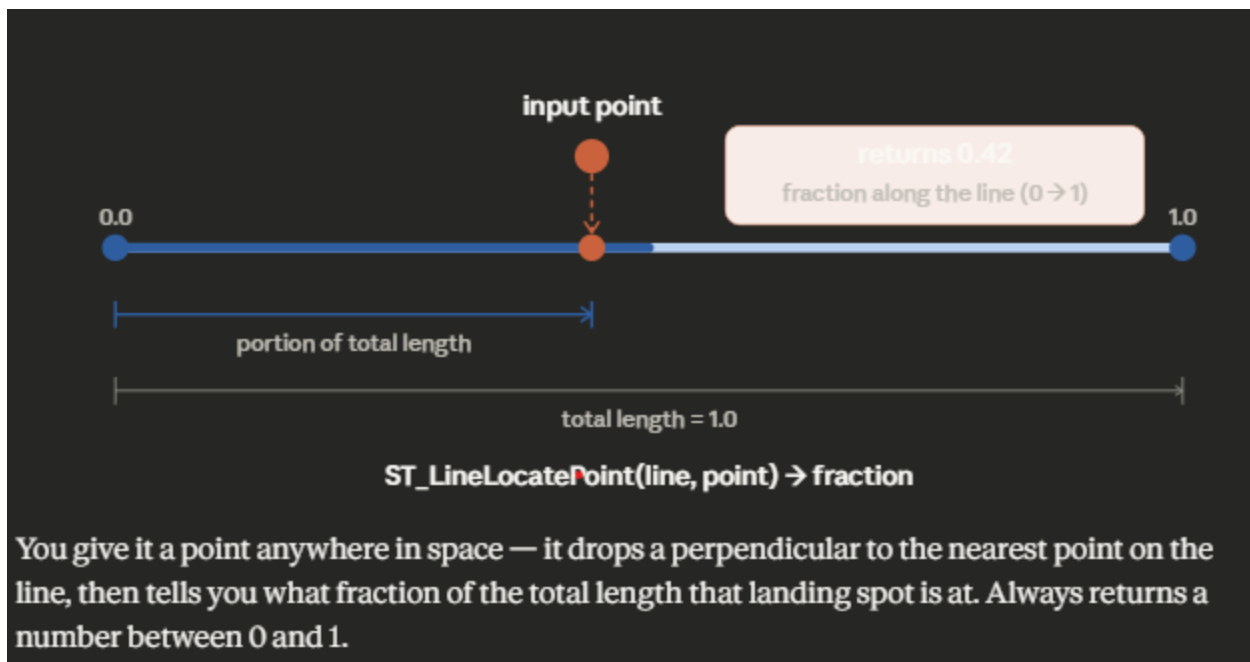
You have:

- 1 A **line layer** → roads
- 2 An **event table** → points or segments along roads

1 ST_LineLocatePoint

Finds where a point lies along a line. Even if point is off the line → it snaps to closest point.

```
SELECT ST_LineLocatePoint(  
    'LINESTRING(0 0, 2 2)',  
    'POINT(1 1)'  
);
```

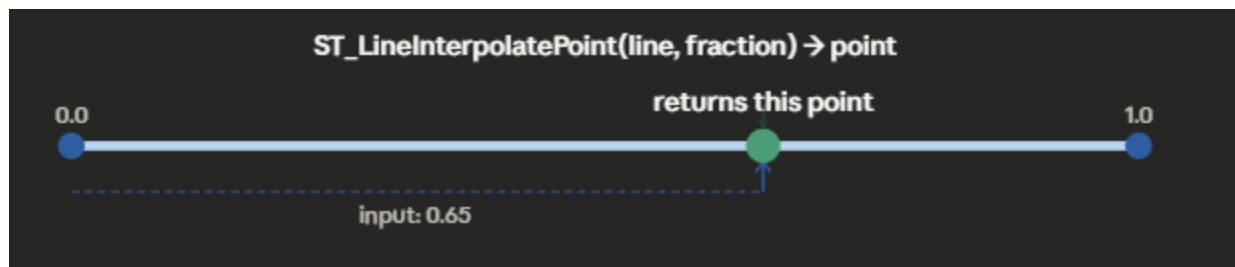


3 ST_LineInterpolatePoint

Converts measure back into a point.

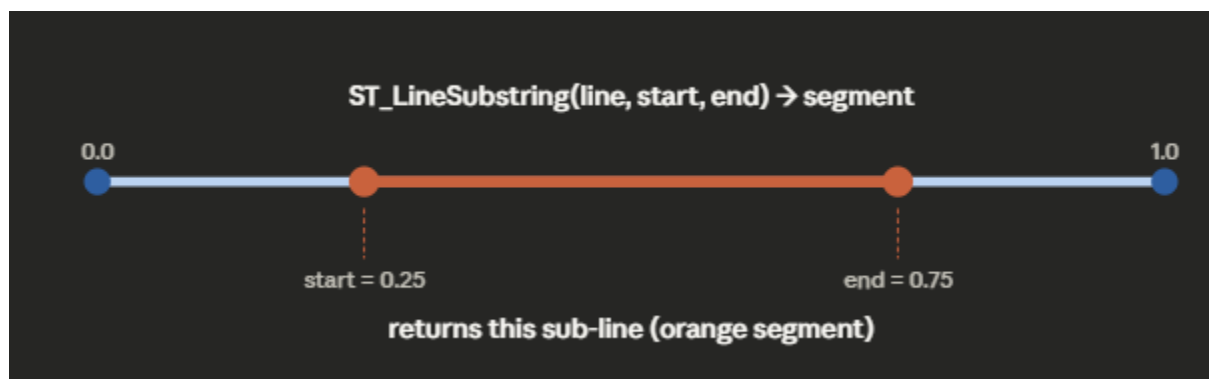
Example:

```
SELECT ST_AsText(  
  ST_LineInterpolatePoint(  
    'LINESTRING(0 0, 2 2)',  
    0.5  
  )  
);
```

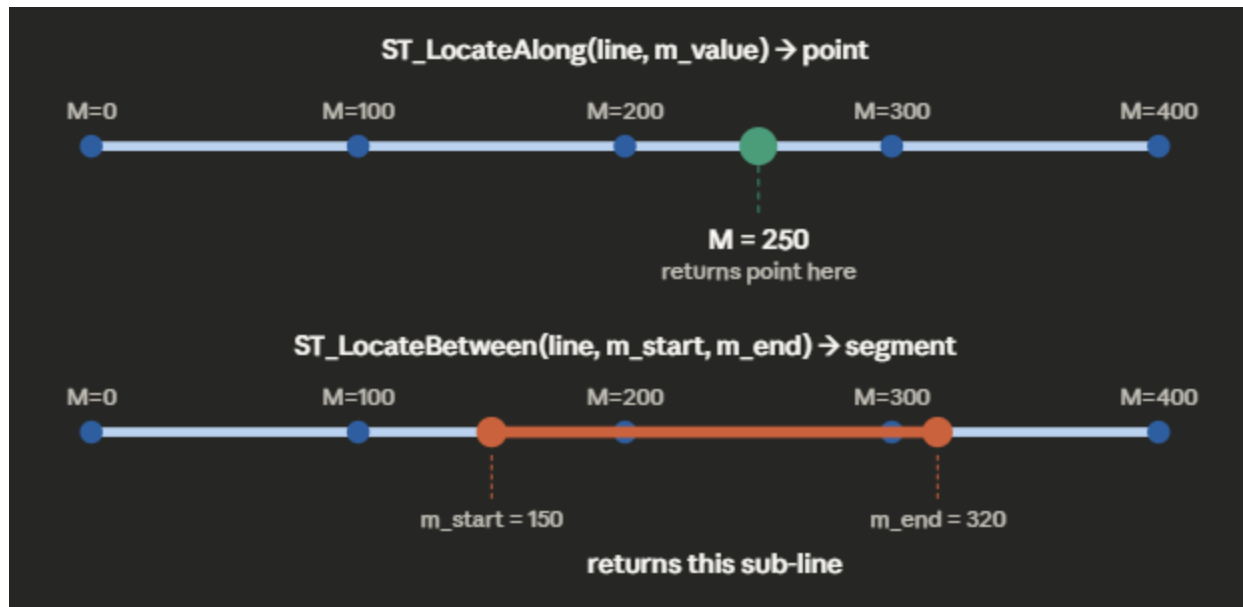


ST_LineSubstring Extract part of line

```
SELECT ST_AsText(ST_LineSubstring('LINESTRING(0 0, 2 2)', 0.25, 0.75))
```



ST_LocateAlong & ST_LocateBetween



```
SELECT ST_AsText(ST_LocateAlong('LINESTRING M(0 0 0, 2 2 400)', 250))
```

```
SELECT ST_AsText(ST_Locatebetween('LINESTRING M(0 0 0, 2 2 400)', 150,320))
```

◆ 2 Create Event Table Example

Convert subway stations into events along streets:

Steps:

1. Find closest street
2. Calculate measure
3. Store reference


```
with near as (  
  SELECT  
  
    streets.gid AS streets_gid,  
    subways.gid AS subways_gid,  
    ST_Distance(streets.geom, subways.geom) AS distance,  
    subways.geom AS subways_geom,  
    ST_GeometryN(streets.geom,1) AS streets_geom  
  
  FROM nyc_streets streets  
    JOIN nyc_subway_stations subways  
      ON ST_DWithin(streets.geom, subways.geom, 100)  
  ORDER BY subways_gid, distance ASC  
)  
SELECT  
  DISTINCT ON (subways_gid)  
    subways_gid,  
    streets_gid,  
    ST_LineLocatePoint(streets_geom, subways_geom) AS measure,  
    distance  
FROM near;
```

Function	Purpose
<code>ST_LineLocatePoint</code>	Get measure along line
<code>ST_LineInterpolatePoint</code>	Convert measure → point
<code>ST_LineSubstring</code>	Extract part of line
<code>ST_LocateAlong</code>	Get geometry at measure
<code>ST_LocateBetween</code>	Get geometry in range
<code>ST_AddMeasure</code>	Add measure dimension

Measure-based (M-values)

`ST_AddMeasure()`

It **adds M values (measure values)** along a geometry.

It is a **third coordinate** attached to a geometry vertex that represents a **position along a line**.

`LINESTRING M (X Y M, X Y M, ...)`

```
SELECT ST_AsText(  
    ST_AddMeasure(  
        'LINESTRING(0 0, 10 0)', 0, 100  
    )  
);
```

```
SELECT  
    gid,  
    ST_ASTEXT(ST_AddMeasure(geom, 0, 1))  
FROM nyc_streets;
```

```
WITH measured_streets AS (  
    SELECT gid, ST_AddMeasure(geom, 0, 1) AS geom_m  
FROM nyc_streets  
)  
select st_astext(ST_LocateAlong(geom_m, 0.25))FROM measured_streets;
```

Find the geometry at between **0.25** and **0.50**.

```
WITH measured_streets AS (  
    SELECT gid, ST_AddMeasure(geom, 0, 1) AS geom_m  
FROM nyc_streets  
)  
  
select st_astext(ST_LocateBetween(geom_m, 0.25, .50))FROM  
measured_streets;
```

PostgreSQL & PostGIS FUNCTIONS

1 What is a Function in PostgreSQL?

A **function** is:

A named, reusable block of SQL (or logic) that **takes input, does work, and returns a result.**

Think of it as:

- a reusable query
- a calculator
- a mini program inside the database

```
CREATE OR REPLACE FUNCTION schema.function_name(  
    param_name param_type  
)  
RETURNS return_type  
LANGUAGE sql | plpgsql  
IMMUTABLE | STABLE | VOLATILE  
STRICT  
AS $$  
    function body  
$$;
```

3 PARAMETERS (Inputs)

```
(neighborhood_name text)
```

This creates a **variable** you can use inside the function.

3 RETURNS (What comes out?)

Scalar (one value)

```
RETURNS boolean
```

```
RETURNS text
```

```
RETURNS double precision
```

Table (multiple rows)

```
RETURNS TABLE (  
    col1 type,  
    col2 type  
)
```

3 Language (?)

✓ Difference Between **LANGUAGE sql** and **LANGUAGE plpgsql**

Think of it like this:

SQL functions = single query
PL/pgSQL functions = mini program

🔴 **LANGUAGE sql**

✓Rules

- ✓ No **IF**
- ✓ No **LOOP**
- ✓ No variables
- ✓ No **BEGIN/END**
- ✓ Very fast

LANGUAGE sql

```
AS $$ SELECT ... $$;
```

● LANGUAGE plpgsql

- ✓ IF / ELSE
- ✓ Loops
- ✓ Variables
- ✓ Multiple statements

```
LANGUAGE plpgsql
```

```
AS $$ BEGIN ... END; $$;
```

SQL-language functions return the result of their final SQL statement automatically.

PL/pgSQL functions require explicit RETURN or RETURN NEXT.

6 VOLATILITY (THIS IS BIG)

“If I run this function twice with the same inputs... can I trust the answer to be the same?”

● IMMUTABLE — = “I trust you forever”

- Same input → same output **forever**
- No table reads
- Safe for indexes

Examples:

```
LOWER(text)
```

```
ST_Length(geom)
```

● **STABLE** — “I trust you for THIS query only”

- Same result **within one query**
- Can read tables
- Cannot be indexed

Examples:

`NOW()`

`SELECT COUNT(*) FROM table`

Most PostGIS query functions = **STABLE**

● **VOLATILE** — “Unpredictable” “I don’t trust you at all”

- Result may change every call
- Writes data
- Uses random/time

Examples:

`random()`

`INSERT / UPDATE`

One Tiny Example (THIS MAKES IT CLICK)

SELECT

```
random(),  
random(),  
random();
```

Every column → different number

 **VOLATILE**

SELECT

```
now(),  
now(),  
now();
```

All columns → same time

 **STABLE**

SELECT

```
lower('HELLO'),
```

```
lower('HELLO'),  
lower('HELLO');
```

Always same

 IMMUTABLE

Why PostgreSQL CARES

Because it wants to be **fast**.

- If it **trusts** the function - compute once
- If it doesn't - compute every time

Wrong trust = wrong results.

STRICT — The “NULL shortcut”

STRICT

Means:

If **any input is NULL**, PostgreSQL **does not run** the function and returns NULL immediately.

Example to get all street names in a neighborhood

```
-- create function
```

```
create or replace function
```

```
public.streets_in_neighborhoods(neighborhood_name text)
```

```
returns table (street_id int,street_name text)

language sql

stable

strict

as $$

select s.id,s.name
from nyc_streets s
join nyc_neighborhoods n
on st_intersects (s.geom,n.geom)
where lower(n.name)=lower(neighborhood_name)

$$

-- test it

select * from streets_in_neighborhoods('east village')
```

you want to classify EACH street inside a neighborhood:

- 1. go inside the neighborhood polygon**
- 2. take street 1 → measure its length inside the neighborhood**
- 3. tag it: major / medium / local**
- 4. move to street 2, repeat...**
- 5. return a table of streets with their clipped length + tag**

```

CREATE OR REPLACE FUNCTION public.tag_streets_in_neighborhood_loop(
    p_neighborhood text
)
RETURNS TABLE (
    neighborhood text,
    street_gid int,
    street_name text,
    length_m double precision,
    tag text
)
LANGUAGE plpgsql
STABLE
STRICT
AS $$
DECLARE
    v_neigh_geom geometry;
    r record;
    v_clip geometry;
    v_len double precision;
BEGIN

-- get the single geometry for one neighborhood in that study
select geom into v_neigh_geom

```

```

from nyc_neighborhoods

where name = p_neighborhood;

-- Loop through all streets that intersects the geom in the db,

for r in
select gid,name,geom
from nyc_streets
where st_intersects (geom,v_neigh_geom)

loop
    -- Clip street to neighborhood boundary
v_clip:= st_intersection(r.geom,v_neigh_geom);

-- measure the lenght
v_len:=st_length(v_clip);

--tag

if v_len>500 then

    tag :='major';

elseif v_len>=250 then

tag :='med size';

```

```
else

tag := 'local';

end if ;

neighborhood := p_neighborhood;

street_gid := r.gid;

street_name := r.name;

length_m := v_len;

return next;

end loop;


RETURN;

END;

$$


--test it

SELECT *

FROM public.tag_streets_in_neighborhood_loop('East Village')

ORDER BY length_m DESC
```



Stored Procedure in PostgreSQL?

A **stored procedure** is a block of SQL / PLpgSQL code that:

- 👉 Runs a sequence of commands
- 👉 Can modify data (INSERT / UPDATE / DELETE / COMMIT)
- 👉 Does **NOT** return a value

```
CREATE OR REPLACE PROCEDURE public.raise_prices(percent_increase
numeric)
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    UPDATE products
```

```
    SET price = price * (1 + percent_increase/100);
```

```
    COMMIT;
```

```
END;
```

```
$$;
```

Memory Trick

FUNCTION → returns data

PROCEDURE → does work

PostgreSQL functions are optimized for set-based queries and expressions, while stored procedures are designed for executing batches of commands and transaction control; neither is inherently faster.

A **function**:

- 👉 Returns a value (or table)
- 👉 Can be used inside **SELECT**, **WHERE**, **JOIN**
- 👉 Usually used for calculations & queries

When to Use FUNCTION

- ✓ Calculations
- ✓ Spatial queries
- ✓ Return values
- ✓ Use inside SELECT
- ✓ PostGIS logic

When to Use PROCEDURE

- ✓ Data migration
- ✓ Batch updates
- ✓ ETL processes
- ✓ Cleanup jobs
- ✓ Administrative tasks

Add new column to the streets and calculate there length

```
alter table nyc_streets  
add column if not exists newlenght double precision;
```

```
create or replace procedure update_street_length()  
LANGUAGE plpgsql  
AS $$  
BEGIN
```

```
update nyc_streets
set newlength = ST_Length(geom);
commit;
end;
$$;
```

```
call update_street_length();
```

```
select newlength from nyc_streets
limit 10
```



What is a Trigger?

A **trigger** is a rule that tells PostgreSQL:

“When THIS event happens on a table, automatically run THIS function.”

You do **not call** triggers yourself.

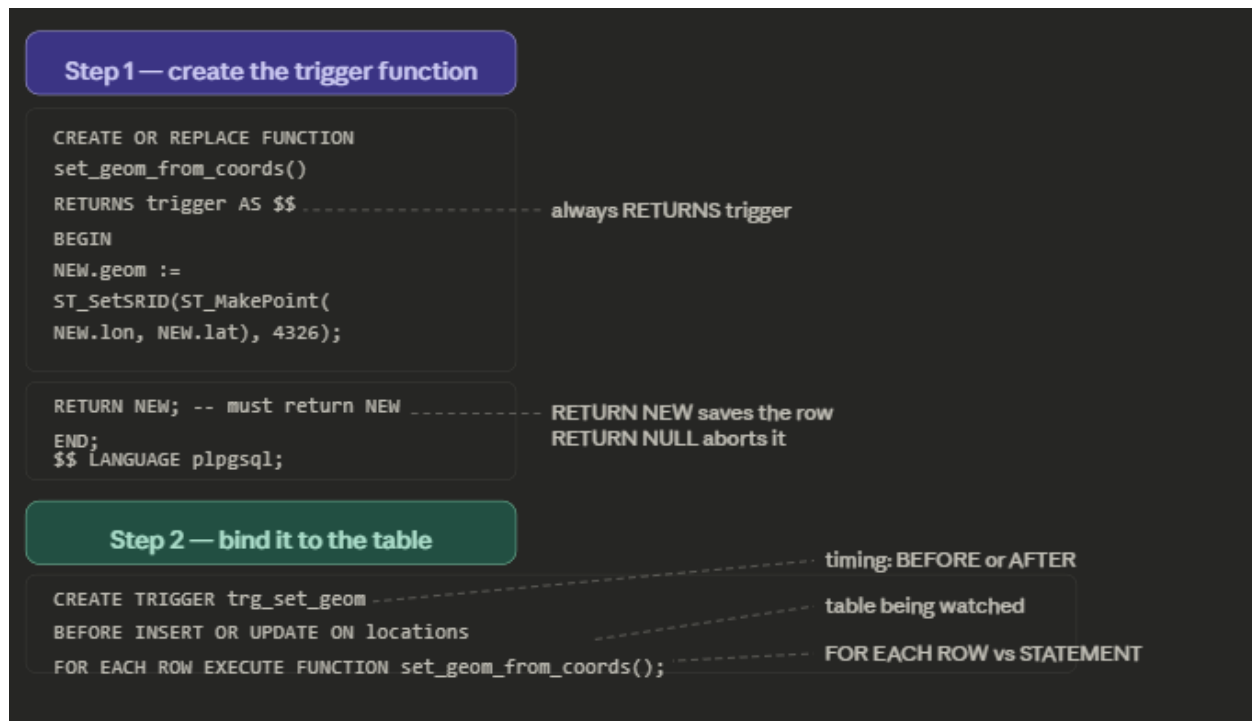
They fire **automatically**.



Trigger Has Two Parts

1) A trigger FUNCTION (special function)

2) A trigger that attaches it to a table



Key Difference in One Sentence

A trigger is an automatic reaction to a table event, while functions and procedures are executed explicitly by the user or application.

Common Trigger Events

BEFORE INSERT

AFTER INSERT

BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE



BEFORE vs AFTER

BEFORE Trigger

Modify data before saving.

```
NEW.column := value;
```

AFTER Trigger

React after row is saved (logging, auditing).

BEFORE triggers modify rows before they are written, while AFTER triggers execute after the row has been committed and are typically used for logging or side effects.



Core Difference (one sentence)

BEFORE = change the row

AFTER = respond to the change

INSERT starts

↓

BEFORE triggers (if any)

↓

Row saved

↓

AFTER trigger runs

↓

Log row inserted

Both triggers can access:

NEW.column -- new row

OLD.column -- old row (for UPDATE/DELETE)

What are NEW and OLD?

Inside a trigger function, PostgreSQL automatically gives you two special row variables:

NEW → the row AFTER change (new version)

OLD → the row BEFORE change (old version)

They represent one row at a time.

Trigger Event	OLD exists?	NEW exists?
INSERT	✗	✓
UPDATE	✓	✓
DELETE	✓	✗

Parcelname = park a >>>> parace lname = park b

Old.col = park a NEW.col park b

old.col = null new.col=park c

old.col=park c new.col= null

Performance Rule

Triggers run **row-by-row**.

If you bulk load 1 million streets → trigger runs 1 million times.

```
-- 1st part the function
create or replace function update_streets_tgr_func()
returns trigger
language plpgsql
as $$
begin
new.newlenght := ST_Length(new.geom);
return new;
end;

$$

-- optional delete the trigger if exists
drop trigger if exists trg_streets_update ON nyc_streets;

-- 2nd part connect the trigger to the function
create trigger trg_streets_update
before insert or update
ON nyc_streets
for each row
EXECUTE FUNCTION update_streets_tgr_func()

-- test it
UPDATE nyc_streets
SET geom = geom
WHERE gid = 1;
select gid,newlenght
```

```
from nyc_streets
where gid=1
```

Rules

🔑 “When someone runs *THIS SQL*, secretly replace it with *THAT SQL* before Postgres executes anything.”

Simple Mental Model

User writes:

```
INSERT INTO mytable VALUES (...);
```

Postgres sees a rule and silently turns it into:

```
INSERT INTO othertable VALUES (...);
```

Make a view writable (classic use)

```
CREATE RULE log_delete AS
```

```
ON DELETE TO products
```

```
DO ALSO
```

```
INSERT INTO delete_log(product_name, deleted_at)
```

```
VALUES (OLD.name, now());
```

Tracking Edit History using Triggers

Every edit creates a new version, old versions are time-stamped, and you can query any moment in history using **valid_range**.

We keep:

- ✓ One **main table** → **nyc_streets** (current streets)
- ✓ One **history table** → **nyc_streets_history** (every version ever)

Triggers automatically copy changes into the history table.

What is **valid_range**

valid_range stores: `[start_time , end_time)`

Each row in **nyc_streets_history** = **one version of one street**

TSTZRANGE is a PostgreSQL data type that stores a range of timestamps with time zone.

Think of it like:

“This record was valid from time A until time B.”

Extra columns:

Column	Meaning
hid	history row id
gid	street id
valid_range	when this version existed TSTZRANGE
created_by	who created this version
deleted_by	who ended this version

Whenever something happens in **nyc_streets**:

Action	Trigger Behavior
INSERT	Add new history row
DELETE	Close existing history row
UPDATE	Close old row + insert new row

Example

--1 create a copy of your data.

```
create table nyc_streets_history as
select * from nyc_streets
```

--2 check the new data

```
select * from nyc_streets_history
limit 1
```

--3 add additional req columns:

```
alter table nyc_streets_history
add column created_by text,
add column deleted_by text,
add column hid SERIAL primary key,
add column valid_range TSTZRANGE
```

--4 update nyc fields

```
update nyc_streets_history
set created_by = current_user,
    valid_range= tstzrange(now(),null)
```

--check

```
select * from nyc_streets_history
limit 1
```

-- now history table is ready to receive the data.

```
CREATE OR REPLACE FUNCTION nyc_streets_insert()
RETURNS trigger AS
```

```
$$
BEGIN

    INSERT INTO nyc_streets_history
    (gid, name, oneway, type,geom,newlenght,valid_range, created_by)
    VALUES
    (NEW.gid, NEW.name, NEW.oneway,new.type, NEW.geom,new.newlenght,
     tstzrange(now(), NULL), current_user);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- attach the trigger to the function
CREATE TRIGGER trg_nyc_street_insert
AFTER INSERT ON nyc_streets
FOR EACH ROW
EXECUTE FUNCTION nyc_streets_insert();

-- test it , insert new line and see what happens
insert into nyc_streets (name)
values ('test')

select * from nyc_streets_history
where name = 'test'
```

--on delete DELETE Trigger

```
CREATE OR REPLACE FUNCTION nyc_streets_delete()
RETURNS trigger AS
$$
BEGIN
    UPDATE nyc_streets_history
    SET valid_range = tstzrange(lower(valid_range), now()),
        deleted_by = current_user
    WHERE gid = OLD.gid
    AND valid_range @> now();

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_nyc_streets_delete
after delete on nyc_streets
for each row
execute function nyc_streets_delete()
--
```

-- test the result

```
delete from nyc_streets
where name = 'test'
```

```
select * from nyc_streets_history
where name = 'test'
```

-- On Update

```
CREATE OR REPLACE FUNCTION nyc_streets_update()
RETURNS trigger AS
$$
BEGIN

    -- close old version
    UPDATE nyc_streets_history
    SET valid_range = tstzrange(lower(valid_range), now()),
        deleted_by = current_user
    WHERE gid = OLD.gid
    AND valid_range @> now();

    -- insert new version
    INSERT INTO nyc_streets_history
    (gid, name, type, geom, valid_range, created_by)
    VALUES
    (NEW.gid, NEW.name, NEW.type, NEW.geom,
     tstzrange(now(), NULL), current_user);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_streets_updates
AFTER UPDATE ON nyc_streets
FOR EACH ROW
EXECUTE FUNCTION nyc_streets_update();
```

```
-- test updating one street
```

```
update nyc_streets  
set name = 'new name'  
where gid=100
```

```
select * from nyc_streets_history  
where gid=100
```

Note the power of this database-backed approach to history: **no matter what tool is used to make the edits, whether the SQL command line, a web-based JDBC tool, or a desktop tool like QGIS, the history is consistently tracked.**

1) All ACTIVE Records (current/live)

```
SELECT *  
FROM nyc_streets_history  
WHERE valid_range @> now();
```

2) All INACTIVE Records (expired)

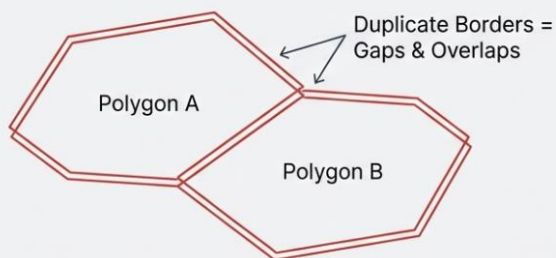
```
SELECT *  
FROM nyc_streets_history  
WHERE NOT (valid_range @> now());
```

PostGIS Topology

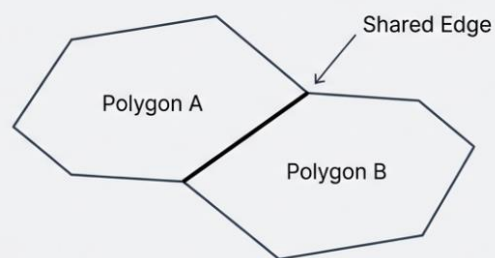
Before writing a single line of SQL, let's understand **why** topology exists and what problem it solves.

The Topology Problem: Shared Boundaries

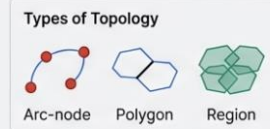
Standard Geometry (Spaghetti Data)



Topological Model



In Topology, the edge is stored once. Both neighbors reference it. Moving the edge updates both neighbors instantly.



The Problem With Regular Geometries

In standard PostGIS (without topology), every polygon stores its own private copy of every border. This seems fine at first — until you need to edit a shared boundary.

✗ Without Topology

✗ Without Topology

- Each polygon has its own copy of borders
- Move one border → neighbor doesn't move
- Result: gaps and overlaps appear instantly
- No automatic consistency guaranteed

✓ With Topology

✓ With Topology

- Neighboring polygons share the exact same edge
- Move that edge once → both polygons update
- No gaps, no overlaps — ever
- All related features stay consistent

🔑 Real-World Use Cases

- **Land parcels** — must never overlap; if one parcel boundary moves, the adjacent parcel must reflect it.
- **Neighborhoods / Boroughs** — smaller areas must tile perfectly to form the larger administrative region.
- **Road networks** — streets must stay connected at intersections; a broken node breaks routing.

Section 2 — The Three Building Blocks

Every shape in a topology is assembled from three **primitive elements**. Think of them like LEGO bricks — simple pieces that combine to form complex structures.

Element	Symbol	What it is
Node	• <code>Point</code>	A corner, endpoint, or junction — a single coordinate pair stored in <code>nyc_topo.node</code>
Edge	— <code>Line</code>	A straight segment connecting exactly two nodes — stored in <code>nyc_topo.edge_data</code>
Face	□ <code>Area</code>	A surface enclosed by edges — stored in <code>nyc_topo.face</code> (as bounding box only)

These three elements have a natural hierarchy: **Nodes** → **Edges** → **Faces** → **Polygons**. Nodes connect to form edges; edges enclose faces; faces combine to represent real-world polygon features.

Section 3 — How Topology Is Stored

When you create a topology called `nyc_topo`, PostGIS creates a dedicated schema with five key tables:

Term	Meaning
<code>edge_data</code>	The actual line segments — the backbone of all boundary information
<code>edge</code>	A convenient view on top of <code>edge_data</code> (same data, easier to query)
<code>node</code>	All endpoints and isolated points
<code>face</code>	Polygon surfaces — but stored only as bounding boxes, not full polygons
<code>relation</code>	The lookup table linking each feature (topogeometry) to its elements

Why does face store only a bounding box?

This is a common point of confusion. The **face table** does not store the complete polygon shape. Instead it stores only the minimum rectangle that contains the face.

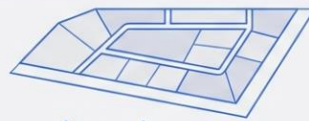
The reason: the true shape is already defined by the edges that surround the face. When PostGIS needs the real polygon, it reconstructs it from those surrounding edges on the fly.

💡 Simple Analogy

Instead of storing a complete drawing of a country, PostGIS stores only its borders (edges) and corners (nodes). From those, it can redraw the country perfectly whenever needed.

Topology Primitives: The LEGO System

TopoGeometry
(The House)



Stored in User Table.

Relation Table

Feature X

→ = Face 44

→ + Edge 12

Primitives
(The Bricks)

Dot
(Node)

Line
(Edge)

Shaded Area
(Face)

Stored in "topo" schema.

Your feature table doesn't store coordinates. It stores a recipe:
"I am made of these existing primitives."

Section 4 — Building a Topology Step **by Step**

Let's walk through the full workflow for a neighborhoods dataset.

Step 1 — Create the Extension and the Topology

```
-- Enable the topology extension
CREATE EXTENSION postgis_topology;

-- Create a named topology with SRID 26918 and 0.5-unit tolerance
SELECT topology.CreateTopology('nyc_topo', 26918, 0.5);

-- Verify it was registered
SELECT * FROM topology.topology;
```

Step 2 — Create Your Table and Add a Topogeometry Column

A topogeometry column does NOT store coordinates. It stores a pointer into the topology system.

```
-- Create the base table
CREATE TABLE nyc_neighborhoods_t (
    boroname  varchar,
    name      varchar,
    PRIMARY KEY (boroname, name)
);

-- Add the topogeometry column (returns a layer_id)
SELECT topology.AddTopoGeometryColumn(
    'nyc_topo',          -- topology name
    'public',            -- schema of the table
    'nyc_neighborhoods_t', -- table name
    'topo',              -- new column name
    'POLYGON'           -- geometry type
);

-- Explore what was registered
SELECT * FROM topology.layer;
```

Step 3 — Populate the Topogeometry Column

The function `topology.toTopoGeom()` converts a regular geometry into a topogeometry by breaking it down into nodes, edges, and faces and storing them in the topology schema.

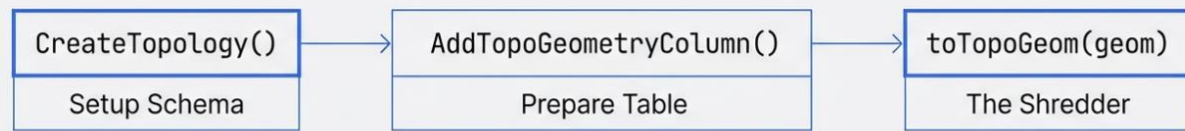
```
INSERT INTO nyc_neighborhoods_t
SELECT boroname, name,
       topology.toTopoGeom(geom, 'nyc_topo', 1)
FROM nyc_neighborhoods
WHERE ST_IsValid(geom);
```

Step 4 — Verify the Topology Was Built

```
-- Count primitives
SELECT 'edge', count(*) FROM nyc_topo.edge
UNION ALL
SELECT 'node', count(*) FROM nyc_topo.node
UNION ALL
SELECT 'face', count(*) FROM nyc_topo.face;

-- Full summary
SELECT * FROM topology.TopologySummary('nyc_topo');
```

Building and Maintaining Topology



Detecting Errors

Topology reveals hidden overlaps in source data.

```
GetTopoGeomElements(topo) ... HAVING COUNT > 1
```

Finds shared faces where none should exist.



Section 5 — Terms That Look Similar but Are NOT the Same

⚠ This section is critical. Several terms in topology sound nearly identical. Mixing them up will lead to wrong queries and real confusion. Read carefully.

Confusion #1 — Element ID vs. TopoID

Element ID (the brick)	TopoID (the house)
Element ID - Identifies a primitive — a single node, edge, or face - Lives inside the topology schema tables e.g. <code>nyc_topo.face</code>, <code>nyc_topo.node</code> - Many different features can share the same element Example: <code>face_id = 44</code> is face #44 in the raw topology	TopoID - Identifies a feature — e.g. a neighborhood, parcel - Lives in your user table: <code>nyc_neighborhoods_t.topo</code> - Each feature has its own unique topoID Example: <code>topo_id = 7</code> is the "Downtown" neighborhood

The connection between them lives in the `relation` table:

```
-- nyc_topo.relation — the bridge between features and primitives
topo_id | element_id | element_type
-----+-----+-----
7       | 44             | 3 (FACE)
7       | 51             | 3 (FACE)
7       | 76             | 3 (FACE)

-- Means: neighborhood #7 (Downtown) is built from faces 44, 51, and 76
```

LEGO Analogy

- **Element ID** = individual LEGO brick
- **Topogeom ID** = the completed LEGO house
- **relation table** = the instruction sheet showing which bricks make which house

Confusion #2 — Face vs. Topogeometry

A **face** is a raw topology primitive — a single enclosed area defined by edges and nodes. A **topogeometry** is a higher-level feature (like a neighborhood) that may be assembled from one or more faces.

Key Distinction

Face → the brick itself. Stored in `nyc_topo.face`. Has a `face_id`.

Topogeometry → the finished shape in your table. Stored as a tuple like `(1,1,7,3)` in `nyc_neighborhoods_t.topo`.

Confusion #3 — Reading the Topogeometry Tuple

When you query the `topo` column, PostGIS returns a tuple like `(1, 2, 3, 3)`. This is not a coordinate — it is an address pointing to where the shape is defined.

Position	Field	Meaning
1st	Topology ID	Points to a row in topology.topology — which named topology this belongs to
2nd	Layer ID	Points to topology.layer — which table and column hold this feature
3rd	Topogeom ID	The ID of this specific feature inside the topology system
4th	Type Code	1 = POINT 2 = LINESTRING 3 = POLYGON 4 = COLLECTION

So (1, 2, 7, 3) means: "Topology #1, Layer #2, Feature #7, type POLYGON." It is an address, not a shape.

To actually see the geometry, cast it:

```
SELECT boroname, topo::geometry
FROM nyc_neighborhoods_t;
```

Confusion #4 — element_id vs. topogeom_id inside GetTopoGeomElements

The function `topology.GetTopoGeomElements(topo)` takes ONE feature row and explodes it into all the primitive elements that make it up.

It returns rows in the form {element_id, element_type}:

```
-- te[1] = element_id    (the ID of the face/edge/node)
-- te[2] = element_type (1=Node, 2=Edge, 3=Face)

SELECT b.name, te
FROM nyc_neighborhoods_t b,
     topology.GetTopoGeomElements(b.topo) AS te;

-- Example output for ONE neighborhood:
-- {44, 3}    → face #44
-- {51, 3}    → face #51
-- {76, 3}    → face #76
```

Why filter with `WHERE te[2] = 3`?

Because `te[2]` is the **element type code**. When checking for overlaps, we care only about shared areas (faces), not shared borders or corners. Type 3 = face. Types 1 and 2 = node and edge, which legitimately share between neighbors.

Section 6 — Detecting and Fixing

Now that the concepts are clear, let's use them to find and fix problems in real data.

Detect Shared Faces (Overlapping Neighborhoods)

If two features claim the same face, they overlap. This query finds all such conflicts:

```

SELECT array_agg(DISTINCT b.name) AS neighborhoods,
       te
FROM nyc_neighborhoods_t b,
     topology.GetTopoGeomElements(b.topo) AS te
GROUP BY te
HAVING count(DISTINCT b.name) > 1;

```

Any row returned means those neighborhoods share a face — an overlap error.

Inspect the Shared Area Geometry

Once you know which face is shared, visualize it with `ST_GetFaceGeometry()`:

```

SELECT array_agg(DISTINCT b.name) AS neighborhoods,
       te,
       topology.ST_GetFaceGeometry('nyc_topo', te[1]) AS geom
FROM nyc_neighborhoods_t b,
     topology.GetTopoGeomElements(b.topo) AS te
WHERE te[2] = 3          -- faces only
GROUP BY te
HAVING count(DISTINCT b.name) > 1;

```

Fix — Remove the Shared Face from One Feature

After identifying which feature incorrectly claims the face, remove it using

`TopoGeom_remElement()`:

```

-- Inspect the specific face first
SELECT topology.ST_GetFaceGeometry('nyc_topo', 83) AS geom;

-- Remove face 83 from 'Eastchester' (the incorrect claimant)
UPDATE public.nyc_neighborhoods_t
SET topo = topology.TopoGeom_remElement(topo, ARRAY[83, 3]::topoelement)
WHERE name = 'Eastchester';

```

💡 What ARRAY[83, 3] means

- **83** → the `element_id` (face number 83)
- **3** → the `element_type` (3 = face)
- Together they form a `topoelement` — the same structure returned by `GetTopoGeomElements`

Section 7 — Quick Reference Cheat

Function Summary

Term	Meaning
<code>CreateTopology()</code>	Creates a new topology schema with a name, SRID, and tolerance
<code>AddTopoGeometryColumn()</code>	Adds a topogeometry column to an existing table; returns the <code>layer_id</code>
<code>toTopoGeom()</code>	Converts a regular geometry into a topogeometry (inserts nodes/edges/faces)
<code>TopologySummary()</code>	Returns a human-readable count of all elements in a topology
<code>GetTopoGeomElements()</code>	Explodes a topogeometry into its constituent element {id, type} pairs

<code>ST_GetFaceGeometry()</code>	Reconstructs the full polygon of a face from its surrounding edges
<code>TopoGeom_remElement()</code>	Removes a specific element from a topogeometry (for fixing overlaps)

Element Type Codes

Code	Type
1	Node (point)
2	Edge (line)
3	Face (area) ← most common
4	Collection

Remember: a topology doesn't store shapes — it stores relationships. Shapes are reconstructed from edges and nodes when you ask for them.

PostGIS Topology Review

Topology in PostGIS is a way to store geometries as shared edges, nodes, and faces so features stay connected and consistent instead of existing as isolated shapes.

We need it to prevent overlaps, gaps, and broken boundaries, and to ensure that when one boundary changes, all related features update together.

✗ Without Topology (regular geometry)

Each polygon stores its own copy of borders.

If you move one border, the neighbor's border does not move → gaps or overlaps appear.

✓ With Topology

Neighboring polygons share the same edge in the database.

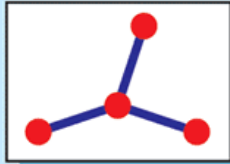
So when you move that edge once, both polygons update automatically.

Real-life examples: land parcels that must not overlap, neighborhoods forming boroughs, and road networks where streets must stay connected.

Types of Topology

Line features can share endpoints

arc-node topology



Area features can overlap with other area features

region topology



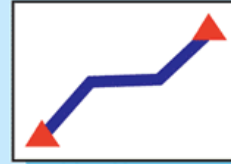
Area features can share boundaries

polygon topology



Line features can share endpoint vertices with point features

node topology



Line features can share segments with other line features

route topology



Point features can share vertices with line features

point events



✓ What are *topology elements*?

Topology elements are the **basic building blocks** used to construct shapes:

1. **Node** → a point (corner, endpoint)
2. **Edge** → a line between two nodes
3. **Face** → an area enclosed by edges

Think:

Nodes make Edges → Edges make Faces → Faces make Polygons

These nodes, edges, and faces live inside the **topology schema** (e.g., `nyc_topo.node`, `nyc_topo.edge`, `nyc_topo.face`).

They are called **topology elements** because they are the primitive components of the topology.

Topology Storage Structure (inside nyc_topo)

- `edge_data` → line segments
- `edge` → view on `edge_data`
- `node` → endpoints & isolated points
- `face` → polygon surfaces (stored as bounding boxes)
- `relation` → links topogeometries to topology elements

Face → polygon surfaces (stored as bounding boxes)

A **face** represents a polygonal area in the topology.

However:

- PostGIS does **not** store the full polygon geometry inside the `face` table.
- It stores only the **bounding box** (minimum rectangle that contains the face).

Why?

Because the **true shape** of the face is defined indirectly by:

- The edges that surround it
- The nodes at edge endpoints

When PostGIS needs the real polygon, it **reconstructs** it from those edges.

Simple analogy

Instead of storing the whole drawing of a country, PostGIS stores:

- The borders (edges)
- The corners (nodes)

From those, it can redraw the country whenever needed.

relation → links topogeometries to topology elements

A **topogeometry** (your feature like a neighborhood or parcel) does NOT store coordinates.

Instead, it stores:

“This feature is made from THESE faces, THESE edges, and THESE nodes.”

That information is kept in the **relation** table.

So the relation table is basically a **lookup table** that says:

Topogeometry A - uses Face 12
Topogeometry A - uses Face 15
Topogeometry A - uses Edge 88

This tells PostGIS exactly **which topology elements** build each feature.

Imagine One Neighborhood Polygon

Visually, the neighborhood looks like a simple square.

But in topology, it is **not stored as a polygon**.

Instead, it is built from:

- **4 Nodes** (corners)
- **4 Edges** (sides)

- **1 Face** (area inside)
-

Step 1 — Nodes (points)

```
N1  -----  N2
|              |
|              |
N4  -----  N3
```

These are just coordinates stored in `nyc_topo.node`.

Step 2 — Edges (lines)

Each edge connects two nodes:

- E1: N1 → N2
- E2: N2 → N3
- E3: N3 → N4
- E4: N4 → N1

Stored in `nyc_topo.edge_data`.

Step 3 — Face (area)

The face says:

“Inside E1, E2, E3, and E4 is one continuous surface.”

Stored in `nyc_topo.face`

(only its **bounding box**, not full polygon).

Step 4 — Topogeometry (the neighborhood feature)

Your table row:

```
Neighborhood = "Downtown"  
Topogeometry ID = 25
```

This row does NOT store coordinates.

Instead, PostGIS writes entries in the **relation table** like:

```
topogeom_id | element_type | element_id  
-----+-----+-----  
25          | face        | 101
```

Meaning:

 Neighborhood 25 is made from **Face 101**

What happens if a border moves?

Suppose Edge E2 is shifted slightly.

Because:

- Face 101 uses E2
- Neighborhood uses Face 101

Result:

- ✓ Neighborhood shape updates
- ✓ Any adjacent neighborhood using the same edge updates
- ✗ No gaps
- ✗ No overlaps

All automatically.

1 'nyc_topo'

Name of the **topology** you already created.

2 'public'

Schema where your table lives.

3 'nyc_neighborhoods_t'

The table that will receive the new column.

4 'topo'

Name of the column to create.

5 'POLYGON'

Type of topogeometry.